

Part 1 – Programming & Computer Science Foundations

Chapter 1: How Software Development Works · Chapter 2: Programming Fundamentals

Chapter 1 – How Software Development Works

LEARNING OBJECTIVES

By the end of this chapter you can explain, in plain language, what software is, how a client talks to a server, what an API and a database are for, and what actually happens between typing a URL and seeing a page. No coding experience is assumed.

1.1 What is software?

Software is a set of instructions that tells a computer what to do. Your phone's camera app, a bank's website, and the code running an ATM are all software — text files full of instructions, translated into something the machine's processor can execute.

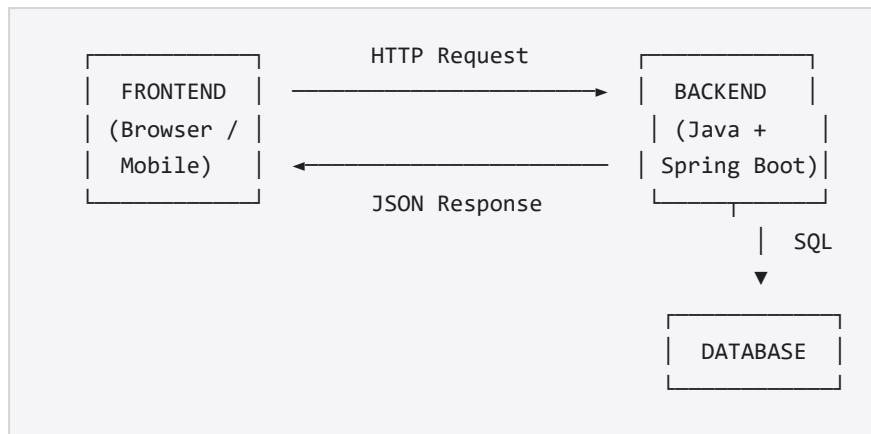
Two broad categories matter for a backend developer:

Category	Definition	Examples
Application software	Software end users interact with directly to accomplish a task	Gmail, Instagram, a food-delivery app
System software	Software that manages hardware and runs other software	Operating systems (Windows, Linux), device drivers, the JVM

1.2 Backend vs. frontend

Frontend is everything the user sees and touches — buttons, screens, forms, animations. It runs in the browser or a mobile app. **Backend** is everything that happens out of sight: business logic, data storage, security, and the rules that decide what the frontend is allowed to show.

This course is entirely about the backend: the Java code, the Spring Boot server, and the database that a mobile app or website talks to behind the scenes.



1.3 Client-server architecture

Almost every application you use follows the **client-server model**. The *client* (a browser, mobile app, or another server) sends a request. The *server* – the backend application you'll build in this course – receives it, does work, and sends back a response.

Key properties of this model:

- The client never touches the database directly – it always goes through the server
- One server can handle requests from thousands of different clients
- The server enforces rules the client can't be trusted to enforce (e.g. "only the account owner can withdraw money")

1.4 APIs

An **API** (Application Programming Interface) is the contract that lets the client and server talk. It defines: what URLs exist, what data to send, and what data comes back. Later in this course you will build REST APIs – the dominant style of API for web and mobile backends – using Spring Boot.

```
GET /api/products/42      → returns product #42 as JSON
POST /api/orders          → creates a new order
{
  "productId": 42,
  "quantity": 3
}
```

1.5 Databases

A **database** is organized, persistent storage. "Persistent" is the key word: unlike a variable in a running program, data in a database survives after the program stops. Part 10 and Part 11 cover relational databases and SQL in depth; for now, know that your Spring Boot application will almost always sit in front of a database like PostgreSQL or MySQL.

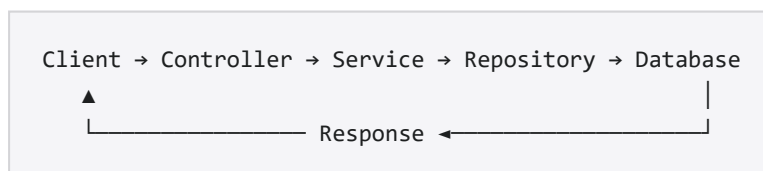
1.6 Runtime, compiler, interpreter, and the JVM ecosystem

Term	Meaning
Compiler	A program that translates your source code into another form before it runs (Java's <code>javac</code> compiles <code>.java</code> to <code>.class</code> bytecode)
Interpreter	A program that reads and executes source code line-by-line, without a separate compile step (used by Python, JavaScript)
Runtime	The environment a program executes in – for Java, this is the JVM (Java Virtual Machine)
JVM	Java Virtual Machine – executes compiled bytecode, providing memory management and platform independence

Java is unusual: it is *both* compiled and interpreted. `javac` compiles your source into bytecode, and the JVM then interprets (and just-in-time compiles) that bytecode into native machine instructions. Chapter 3 covers this pipeline in full detail.

1.7 How a web application works – request/response lifecycle

1. User types a URL or taps a button in the app (the **client**)
2. **DNS** resolves the domain name to a server IP address
3. The client opens a connection and sends an **HTTP request**
4. The server's Spring Boot application receives the request in a **Controller**
5. The Controller calls a **Service** that holds business logic
6. The Service calls a **Repository** to read/write the **Database**
7. The Service returns a result to the Controller
8. The Controller sends back an **HTTP response**, usually JSON
9. The client renders the result to the user



REAL-WORLD INSIGHT

This Controller → Service → Repository chain is the backbone of nearly every Spring Boot application you'll ever build. Every project in this course (Part 35) follows it. Learn this shape now and the rest of the course becomes "filling in the boxes."

1.8 A day in the life of a backend developer

A realistic day mixes: reading a ticket describing a bug or feature, writing code in an IDE, running tests locally, committing to Git, opening a pull request, responding to code review comments, and occasionally debugging a production issue using logs and monitoring dashboards. Parts 15 (Git), 21 (Testing), 27 (CI/CD) and 37 (Debugging) build these exact skills.

INTERVIEW QUESTIONS – CHAPTER 1

1. Explain the client-server model in one sentence.
2. What is the difference between an API and a database?
3. Walk through what happens between typing a URL and seeing a webpage.
4. What is the difference between a compiler and an interpreter?
5. Why can't a mobile app talk directly to a database?

Chapter 1 Summary

- Software = instructions; backend = logic + data hidden from the user
- Client-server: clients request, servers respond, databases persist
- APIs are the contract between client and server
- Java is compiled to bytecode, then run by the JVM
- Every request flows Controller → Service → Repository → Database and back

Chapter 2 – Programming Fundamentals

LEARNING OBJECTIVES

Understand variables, data types, operators, conditionals, loops and functions – the building blocks of every program, taught here in plain pseudocode before Chapter 3 introduces real Java syntax.

2.1 Variables and constants

A **variable** is a named box that holds a value which can change. A **constant** is a named box whose value is fixed once set. Naming a value lets you reuse and reason about it instead of repeating literal data everywhere.

```
// pseudocode
age = 25           // variable – can change later
PI = 3.14159       // constant – should never change
```

2.2 Data types

Type	Holds	Example
Integer	Whole numbers	42, -7, 0
Floating-point	Decimal numbers	3.14, -0.5
Boolean	True or false	true, false
Character	A single letter/symbol	'A', '\$'
String	Text (sequence of characters)	"Hello, World"

2.3 Operators and expressions

An **operator** performs an action on values (**operands**). An **expression** is any combination of values and operators that produces a result.

Category	Operators	Example
Arithmetic	+ - * / %	7 % 2 → 1(remainder)
Comparison	== != > < >= <=	5 > 3 → true
Logical	AND OR NOT	true AND false → false

2.4 Input, output, and type conversion

Programs read input (from a keyboard, a file, a network request) and produce output (to a screen, a file, a response). **Type conversion** changes a value from one type to another – e.g. turning the text "25" into the number 25 so it can be used in arithmetic.

2.5 Conditional statements

Conditionals let a program make decisions.

```
if (age >= 18) {
    print("Adult")
} else {
    print("Minor")
}
```

2.6 Loops

Loops repeat a block of instructions. A **for** loop repeats a known number of times; a **while** loop repeats while a condition holds; **nested loops** are loops inside loops, used for grids, tables, and comparing every pair of items.

```
for i from 1 to 5:
    print(i)           // 1 2 3 4 5

count = 0
while count < 3:
    print(count)
    count = count + 1
```

2.7 Functions and scope

A **function** (called a *method* in Java) is a named, reusable block of instructions that can take inputs (parameters) and produce an output (return value). **Scope** determines where a variable can be seen and used – a variable declared inside a function normally can't be accessed outside it.

```
function add(a, b):  
    return a + b  
  
result = add(3, 4)    // result = 7
```

2.8 Recursion

A function that calls itself is **recursive**. Every recursive function needs a **base case** (a condition that stops the recursion) or it will run forever.

```
function factorial(n):  
    if n == 0: return 1        // base case  
    return n * factorial(n-1)  // recursive case  
  
factorial(4) → 4 * 3 * 2 * 1 * 1 → 24
```

COMMON MISTAKE

Forgetting the base case in a recursive function causes infinite recursion, which crashes the program with a stack overflow. Always write the base case first, then the recursive case.

2.9 Debugging fundamentals

Debugging is the systematic process of finding why a program doesn't behave as expected: reproduce the problem reliably, narrow down where it happens (add print statements or use a debugger to inspect variables), form a hypothesis, test it, and fix the smallest thing that resolves the root cause — not just the symptom.

PRACTICE NOW – 60 FOUNDATIONAL EXERCISES

Solve each in pseudocode first, then again in Java once you finish Chapter 6. Grouped by topic.

Variables & types (1–10): 1) Swap two variables without a third. 2) Convert a temperature from Celsius to Fahrenheit. 3) Compute the area of a circle. 4) Compute the area and perimeter of a rectangle. 5) Convert seconds into hours/minutes/seconds. 6) Check if a number is even or odd. 7) Find the largest of three numbers. 8) Calculate simple interest. 9) Convert inches to centimeters. 10) Compute a percentage.

Conditionals (11–20): 11) Classify a triangle by its sides. 12) Grade a score into A/B/C/D/F. 13) Check if a year is a leap year. 14) Determine if a number is positive, negative, or zero. 15) Check if a character is a vowel. 16) Determine eligibility to vote. 17) Find the maximum of four numbers. 18) Check if a triangle is valid given three sides. 19) Compute a discount tier based on purchase amount. 20) Determine BMI category.

Loops (21–35): 21) Print numbers 1–100. 22) Sum numbers 1–N. 23) Print a multiplication table. 24) Reverse the digits of a number. 25) Check if a number is prime. 26) Print all primes up to N. 27) Compute the factorial of N. 28) Print the Fibonacci sequence up to N terms. 29) Check if a number is a palindrome. 30) Count vowels in a string. 31) Print a right triangle of stars. 32) Print a pyramid of stars. 33) Sum the digits of a number. 34) Find the GCD of two numbers. 35) Find the LCM of two numbers.

Functions & recursion (36–45): 36) Write a function to check primality. 37) Write a function that reverses a string. 38) Write a recursive factorial function. 39) Write a recursive Fibonacci function. 40) Write a function computing `power(base, exponent)` recursively. 41) Write a function that finds the maximum of an array. 42) Write a function that counts occurrences of a character in a string. 43) Write a recursive function to sum digits. 44) Write a function that checks if a string is a palindrome. 45) Write a function computing GCD recursively.

Mixed / mini-challenges (46–60): 46) FizzBuzz (1–100). 47) Count how many numbers between 1–1000 are divisible by 7 but not 5. 48) Simulate a basic calculator with `+`, `-`, `*`, `/`. 49) Find the second-largest number in a list. 50) Sort three numbers ascending without a library sort. 51) Print all Armstrong numbers up to 1000. 52) Convert decimal to binary. 53) Convert binary to decimal. 54) Check if two strings are anagrams. 55) Count word frequency in a sentence. 56) Find the missing number in a sequence 1..N. 57) Rotate an array left by K positions. 58) Find all pairs in an array that sum to a target. 59) Implement a simple guessing-number game. 60) Compute the running average of a stream of numbers.

Chapter 2 Summary & Quick Revision

Concept	One-line takeaway
Variable vs constant	Variables change, constants don't
Conditionals	if / else-if / else branch on a boolean condition
Loops	for = known count, while = condition-driven
Function	Named, reusable block with inputs and a return value
Recursion	A function calling itself – always needs a base case

Next: Part 2 takes these same ideas and expresses them in real Java syntax – JDK/JRE/JVM, your first program, and the complete Java operator and control-flow set.

Part 2 – Java Introduction & Syntax

Chapter 3: Java Introduction · Chapter 4: Java Syntax · Chapter 5: Operators · Chapter 6: Control Flow

Chapter 3 – Java Introduction

3.1 History and features

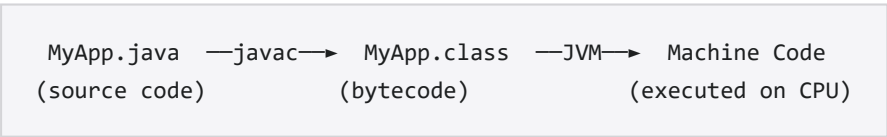
Java was released by Sun Microsystems in 1995, built around the promise "write once, run anywhere." It is statically typed, object-oriented, garbage-collected, and runs on the JVM – meaning the same compiled code runs unmodified on Windows, Linux, or macOS. This course targets **Java 21 LTS**, the current long-term-support release, and **Spring Boot 3.x**.

3.2 JDK, JRE, JVM

Term	Contains	Used for
JVM	The engine that executes bytecode	Running Java programs
JRE	JVM + standard libraries	Running Java programs (no compiler)
JDK	JRE + compiler (javac) + dev tools	Writing and building Java programs

As a developer you install the **JDK** – it's a superset that includes everything needed to both write and run Java.

3.3 Compilation pipeline



javac compiles human-readable source into **bytecode** – a platform-neutral instruction set. The JVM then interprets that bytecode, and its Just-In-Time (JIT) compiler translates hot code paths into native machine instructions for speed. Because bytecode (not native code) is what ships, the same .class file runs on any machine with a JVM – this is platform independence.

3.4 Setting up your environment

Install a JDK 21 distribution (Eclipse Temurin or Oracle JDK), then an IDE. **IntelliJ IDEA Community Edition** is the industry-standard choice for Java and Spring Boot work and is used throughout this course; VS Code with the "Extension Pack for Java" and Eclipse are valid alternatives.

3.5 Your first Java program

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Line by line:

- `public class HelloWorld` – declares a class named `HelloWorld`. The filename must match: `HelloWorld.java`.
- `public static void main(String[] args)` – the entry point the JVM looks for. `public` so the JVM can call it from outside the class; `static` so it runs without creating an object first; `void` because it returns nothing; `String[] args` holds command-line arguments.
- `System.out.println(...)` – calls the standard output stream's print-line method.

Compile and run from a terminal:

```
javac HelloWorld.java    # produces HelloWorld.class  
java HelloWorld          # runs it → prints "Hello, World!"
```

DEBUG THIS

Error: Error: Could not find or load main class HelloWorld. Cause: you ran `java` from a different folder than the `.class` file, or the filename doesn't match the public class name exactly (case-sensitive). Fix: `cd` into the folder containing the `.class` file and confirm filename ↔ class name match.

Chapter 4 – Java Syntax

4.1 Classes and objects, at the syntax level

A **class** is a blueprint; an **object** is an instance built from that blueprint. Full depth on this is Part 3 (OOP) – here we only need enough syntax to read and write basic programs.

```
public class Car {
    String color;           // field
    int speed;

    void accelerate() {     // method
        speed = speed + 10;
    }
}

Car sc-camel-my-car = new Car();    // object creation
myCar.color = "Red";
myCar.accelerate();
```

4.2 Primitive vs. reference types

Primitive type	Size	Example
byte	8-bit	byte b = 100;
short	16-bit	short s = 30000;
int	32-bit	int i = 42;
long	64-bit	long l = 1000000000000L;
float	32-bit decimal	float f = 3.14f;
double	64-bit decimal	double d = 3.14159;
char	16-bit Unicode char	char c = 'A';
boolean	true/false	boolean flag = true;

Everything else – `String`, arrays, and every class you write – is a **reference type**: the variable holds a reference (pointer) to an object on the heap, not the data itself. Assigning one reference variable to another copies the reference, not the object.

```
int a = 5;
int b = a;    // b is an independent copy: 5

int[] arr1 = {1, 2, 3};
int[] arr2 = arr1;    // arr2 points to the SAME array
arr2[0] = 99;
System.out.println(arr1[0]);    // prints 99 – same underlying array
```

4.3 Literals, keywords, identifiers, comments

- **Literal** – a fixed value written directly in code: 42, "hi", true, 'A'.
- **Keyword** – a reserved word with special meaning (class, if, return, static – 50+ total, cannot be used as a variable name).
- **Identifier** – a name you choose for a class, method, or variable.
- **Comments** – // single line, /* multi-line */, /** Javadoc */.

4.4 Naming conventions

Element	Convention	Example
Class	UpperCamelCase	OrderService
Method / variable	lowerCamelCase	calculateTotal, orderId
Constant	UPPER_SNAKE_CASE	MAX_RETRIES
Package	all lowercase, reverse domain	com.example.orders

Chapter 5 – Operators

Category	Operators	Notes
Arithmetic	+ - * / % ++ --	Integer division truncates: $7/2 \rightarrow 3$
Assignment	= += -= *= /= %=	<code>x += 5</code> means <code>x = x + 5</code>
Relational	== != > < >= <=	Use <code>.equals()</code> to compare object content, not <code>==</code>
Logical	&& !	&& and short-circuit: the right side is skipped if the result is already known
Bitwise	& ^ ~ << >> >>>	Rare in everyday backend code; common in low-level/interview questions
Ternary	condition ? a : b	Compact if/else expression

```
int age = 20;
String status = (age >= 18) ? "Adult" : "Minor";

boolean sc-camel-can-vote = age >= 18 && hasCitizenship;    // short-circuits:
hasCitizenship                                              // is never evaluated if age < 18
```

COMMON MISTAKE

`if (name == "Alice")` compares object references, not text content – with `String` objects this can silently fail. Always use `name.equals("Alice")`, or `Objects.equals(a, b)` when either side might be `null`.

Chapter 6 – Control Flow

6.1 if / else-if / else

```
int score = 82;
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else {
    System.out.println("C or below");
}
```

6.2 switch (classic and modern arrow form)

```
// Classic
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Other");
}

// Modern (Java 14+) – no fall-through, returns a value
String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Other";
};
```

6.3 Loops: for, while, do-while, enhanced for

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

int i = 0;
while (i < 5) { System.out.println(i); i++; }

int j = 0;
do { System.out.println(j); j++; } while (j < 5);    // runs body at least once

int[] nums = {10, 20, 30};
for (int n : nums) {                               // enhanced for – reads each element
    System.out.println(n);
}
```

6.4 break, continue, nested loops

```
outer:
for (int a = 0; a < 3; a++) {
    for (int b = 0; b < 3; b++) {
        if (b == 1) continue;           // skip rest of inner loop this iteration
        if (a == 2) break outer;        // exit BOTH loops using a label
        System.out.println(a + "," + b);
    }
}
```

PRACTICE NOW – 40 CONTROL-FLOW EXERCISES

1) FizzBuzz using switch. 2) Print a number pyramid. 3) Print all leap years in a range. 4) Simulate a traffic light state machine with switch. 5) Find all Armstrong numbers 1–9999. 6) Print a diamond pattern of stars. 7) Compute nCr using nested loops. 8) Print a Pascal's triangle of N rows. 9) Sum all multiples of 3 or 5 below 1000. 10) Reverse an array in place using a loop.

11) Find duplicate elements in an array using nested loops. 12) Implement bubble sort. 13) Implement selection sort. 14) Search for a target in an array (linear search). 15) Print a checkerboard pattern. 16) Simulate a simple ATM menu with switch. 17) Convert a grade-point average to a letter grade using nested if. 18) Print all prime factors of a number. 19) Count digits in a number using a while loop. 20) Simulate a login retry loop with a max of 3 attempts.

21) Print the sum of each row and column in a 2D grid. 22) Transpose a 2D matrix. 23) Rotate a matrix 90 degrees. 24) Check if a matrix is symmetric. 25) Implement a basic multiplication table generator taking N as input. 26) Simulate dice rolls and count frequency of each face over 1000 rolls. 27) Print every third character of a string using a loop. 28) Validate a password against multiple nested rules. 29) Find the longest run of consecutive identical characters in a string. 30) Implement insertion sort.

31) Simulate a vending machine state flow with nested switch/if. 32) Compute compound interest year by year in a loop, printing each year's balance. 33) Detect if a sequence of brackets is balanced using a stack (preview of Part 32). 34) Print all Pythagorean triples up to N . 35) Simulate a basic quiz game with a loop asking 5 questions and tallying a score. 36) Build a simple text-based number-guessing game with hints ("higher"/"lower"). 37) Print a spiral pattern in a grid. 38) Implement a basic Caesar cipher using loops. 39) Count how many vowels vs consonants in a paragraph. 40) Simulate a simplified traffic simulation looping through light-state changes.

INTERVIEW QUESTIONS – PART 2

1. What is the difference between JDK, JRE, and JVM?
2. Why is Java called "platform independent"? What actually crosses platforms – source or bytecode?
3. What's the difference between `==` and `.equals()` for objects?
4. Difference between `while` and `do-while`?
5. What does short-circuit evaluation mean for `&&` and `||`?
6. When would you use a labeled `break`?
7. Why doesn't classic `switch` need explicit types beyond `int/String/enum`, and why is `break` required?

Part 2 Summary & Quick Revision

Concept	Key syntax
Program entry point	<code>public static void main(String[] args)</code>
Primitive vs reference	Primitives copy value; references copy the pointer
Modern switch	<code>case X -> value;</code> , no fall-through
Enhanced for	<code>for (Type item : collection)</code>

Next: Part 3 covers Object-Oriented Programming in depth – classes, encapsulation, inheritance, polymorphism, abstraction, and SOLID principles.

Part 3 – Object-Oriented Programming

Classes & Objects · Encapsulation · Inheritance · Polymorphism · Abstraction · Advanced OOP & SOLID

Chapter 7 – Classes & Objects

Everything in Java is modeled as an object built from a class. A **class** defines **state** (fields) and **behavior** (methods); an **object** is a concrete instance with its own copy of that state.

```
public class BankAccount {
    private String owner;           // field (state)
    private double balance;

    public BankAccount(String owner, double balance) { // constructor
        this.owner = owner;         // 'this' distinguishes the field from the parameter
        this.balance = balance;
    }

    public void deposit(double amount) { // method (behavior)
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public double getBalance() { return balance; }
}

BankAccount acc = new BankAccount("Asha", 1000.0);
acc.deposit(500);
System.out.println(acc.getBalance()); // 1500.0
```

this refers to the current object. **static** members belong to the class itself, shared across all instances, rather than to any one object:

```
public class Counter {
    static int sc-camel-total-created = 0; // shared by every instance
    int id;

    Counter() {
        totalCreated++;
        id = totalCreated;
    }
}
```

Chapter 8 – Encapsulation

Encapsulation bundles data with the methods that operate on it, and restricts direct access to that data from outside the class – protecting the object's internal consistency.

Modifier	Visible from
private	Same class only
(package-private / default)	Same package
protected	Same package + subclasses
public	Everywhere

```
public class Person {
    private int age;                // hidden from outside

    public int getAge() { return age; } // controlled read access
    public void setAge(int age) {
        if (age < 0) throw new IllegalArgumentException("Age can't be negative");
        this.age = age;            // controlled write access – validation lives
    here
    }
}
```

Records (Java 16+) generate an immutable, encapsulated data holder in one line – constructor, accessors, equals, hashCode, and toString are all generated:

```
public record Point(int x, int y) {}

Point p = new Point(3, 4);
p.x();           // 3 – accessor is x(), not getX()
// p.x = 5;      // won't compile – records are immutable
```

PRO TIP

Default to `private` fields and expose only what callers genuinely need. An immutable class (all fields `final`, no setters) is inherently thread-safe and easier to reason about – prefer it whenever the object represents a value rather than something that changes over time.

Chapter 9 – Inheritance

Inheritance lets a class (subclass) reuse and extend another class's (superclass) fields and methods – an IS-A relationship: a `Dog` IS-A `Animal`.

```
public class Animal {
    protected String name;
    public Animal(String name) { this.name = name; }
    public void makeSound() { System.out.println(name + " makes a sound"); }
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name);           // calls Animal's constructor first
    }
    @Override
    public void makeSound() {    // overriding – replaces parent behavior
        System.out.println(name + " barks");
    }
}
```

Constructor chaining: every subclass constructor implicitly (or explicitly via `super(...)`) calls a superclass constructor first, so the object is fully initialized top-down.

THINK LIKE A DEVELOPER – COMPOSITION VS. INHERITANCE

Inheritance is powerful but rigid: it locks a subclass to one parent's implementation forever.

"Favor composition over inheritance" means: when in doubt, give a class a reference to another object (has-a) rather than extending it (is-a). A `Car` should *have an Engine*, not *be an Engine*. Spring Beans lean almost entirely on composition via dependency injection – Part 16 builds on this directly.

Chapter 10 – Polymorphism

Type	Also called	Mechanism
Compile-time polymorphism	Method overloading	Same method name, different parameter list, resolved at compile time
Runtime polymorphism	Method overriding	Subclass replaces a method; the JVM picks the actual object's version at runtime

```
// Overloading – same name, different signatures
void print(int x) { System.out.println("int: " + x); }
void print(String s) { System.out.println("string: " + s); }

// Overriding + dynamic dispatch
Animal a = new Dog("Rex");    // declared type Animal, actual type Dog
a.makeSound();                // prints "Rex barks" – the JVM dispatches to Dog's version
```

This last line is **dynamic method dispatch**: the JVM looks at the object's real, runtime type – not the variable's declared type – to decide which overridden method runs. This is the mechanism behind Spring's ability to inject any implementation of an interface without the calling code changing.

Chapter 11 – Abstraction

```
abstract class Shape {
    abstract double area();           // no body – subclasses must implement
    void describe() {                 // concrete method, shared by all shapes
        System.out.println("Area: " + area());
    }
}

interface Payable {
    double calculatePay();
    default void printPaySlip() {      // default method – has a body
        System.out.println("Pay: " + calculatePay());
    }
    static Payable zero() { return () -> 0.0; } // static interface method
}
```

	Abstract class	Interface
Fields	Any (instance state allowed)	Only <code>public static final</code> constants
Constructors	Yes	No
Multiple inheritance	A class extends only one	A class implements many
Use when	Sharing state and partial implementation among closely related classes	Defining a capability/contract unrelated classes can fulfill

A **functional interface** has exactly one abstract method, making it targetable by a lambda expression (Part 6): `Runnable`, `Comparator<T>`, and `Payable` above all qualify.

Chapter 12 – Advanced OOP & SOLID

Relationship	Meaning	Example
Association	Two classes know about each other, independent lifecycles	Teacher ↔ Student
Aggregation	"Has-a" but the part can outlive the whole	Department has Employees
Composition	"Has-a" and the part cannot outlive the whole	House has Rooms – rooms don't exist without the house

Coupling is how much one class depends on another's internals – aim low. **Cohesion** is how focused a single class's responsibility is – aim high. These two ideas underlie the five SOLID principles:

Letter	Principle	In one sentence
S	Single Responsibility	A class should have exactly one reason to change
O	Open/Closed	Open for extension, closed for modification – add behavior via new code, not by editing existing working code
L	Liskov Substitution	A subclass must be usable anywhere its parent is, without breaking correctness
I	Interface Segregation	Prefer several small, specific interfaces over one large, general one
D	Dependency Inversion	Depend on abstractions(interfaces), not concrete implementations

```
// Violates Dependency Inversion – hard-wired to a concrete class
class OrderService {
    private final MySqlOrderRepository repo = new MySqlOrderRepository();
}

// Follows Dependency Inversion – depends on an interface, implementation is injected
class OrderService {
    private final OrderRepository repo; // interface
    OrderService(OrderRepository repo) { this.repo = repo; }
}
```

REAL-WORLD INSIGHT

Spring's entire dependency-injection model (Part 16) is Dependency Inversion applied automatically: you write against interfaces like `OrderRepository`, and Spring wires in the concrete implementation (e.g. a JPA-backed one) at startup. Understanding "D" in SOLID is what makes Spring's IoC container click.

INTERVIEW QUESTIONS – PART 3

1. What is the difference between overloading and overriding?
2. Why can't static methods be overridden (only hidden)?
3. Explain dynamic method dispatch with an example.
4. When would you choose an abstract class over an interface?
5. What is the Liskov Substitution Principle, and give an example that violates it.
6. Difference between composition and aggregation?
7. Why does Java support only single class inheritance but multiple interface implementation?
8. What does `super()` do, and what happens if you omit it?
9. Why are Java records considered good practice for DTOs?
10. Explain the Dependency Inversion Principle and how Spring applies it.

MINI CHALLENGE

Model a **Library** system: an abstract `LibraryItem` (`Book`, `DVD`, `Magazine` subclasses) with overridden `getLoanPeriodDays()`; a `Member` class composed of a list of borrowed items; a `Librarian` service that checks items in/out. Apply encapsulation (private fields, validated setters), inheritance, and at least one interface (`Renewable` with a default method).

Part 3 Summary & Quick Revision

Pillar	Key mechanism
Encapsulation	private fields + public getters/setters, records for pure data
Inheritance	extends, super(), IS-A
Polymorphism	Overloading (compile-time), overriding (runtime, dynamic dispatch)
Abstraction	abstract classes (partial shared state), interfaces (pure contract)
SOLID	Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion

Next: Part 4 — Java Core Deep Dive: Strings, Arrays, the Collections Framework, and Generics.

Part 4 – Java Core Deep Dive

Strings · Arrays · The Collections Framework · Generics

Strings

A `String` in Java is **immutable** – once created, its contents never change. Every "modifying" operation returns a brand-new `String`.

```
String s = "hello";
s.toUpperCase();      // returns "HELLO" – does NOT change s
System.out.println(s); // still "hello"
s = s.toUpperCase();   // now s refers to the new object
```

String literals live in the **string pool**, a special memory region that reuses identical literals:

```
String a = "test";
String b = "test";
System.out.println(a == b);           // true – same pooled object

String c = new String("test");
System.out.println(a == c);           // false – c is a distinct heap object
System.out.println(a.equals(c));      // true – same content
```

Because Strings are immutable, repeated concatenation in a loop creates many throwaway objects – use `StringBuilder` (not thread-safe, fast) or `StringBuffer` (thread-safe, slower) instead.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i).append(","); // mutates the same buffer – O(1) amortized per append
}
String result = sb.toString();
```

COMMON MISTAKE

`String result = ""; for (...) { result += item; }` is $O(n^2)$ – each `+=` creates a new `String` and copies everything so far. Always accumulate with `StringBuilder` inside a loop.

Arrays

```
int[] nums = new int[5];           // fixed size, default 0s
int[] scores = {90, 85, 77};       // array literal
int[][] grid = new int[3][3];      // 2D array (matrix)
grid[1][2] = 7;

for (int i = 0; i < scores.length; i++) {
    System.out.println(scores[i]);
}
```

Arrays have **fixed size** once created — this is exactly why the Collections Framework exists: to provide resizable, richer data structures on top of arrays.

The Collections Framework

The Collections Framework is a unified set of interfaces and implementations for storing and manipulating groups of objects. Three core interfaces:

Interface	Ordering	Duplicates?	Key implementations
List	Insertion order, indexed	Yes	ArrayList, LinkedList, Vector
Set	Varies by impl.	No	HashSet, LinkedHashSet, TreeSet
Map	Key → value pairs	Keys unique	HashMap, LinkedHashMap, TreeMap

List implementations

	Backing structure	get(i)	add at end	insert/remove middle	Use when
ArrayList	Resizable array	O(1)	O(1) amortized	O(n)	Frequent random reads – the default choice
LinkedList	Doubly linked list	O(n)	O(1)	O(1) at known node	Frequent insert/remove at both ends (also implements Deque)
Vector	Resizable array, synchronized	O(1)	O(1) amortized	O(n)	Legacy – avoid in new code, use ArrayList + explicit sync if needed

```
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.remove("Ravi");
for (String n : names) System.out.println(n);
```

Set implementations

	Ordering	Backed by	Notes
HashSet	None	HashMap	$O(1)$ average add/contains; fastest general-purpose set
LinkedHashSet	Insertion order	Hash table + linked list	Predictable iteration order at a slight memory cost
TreeSet	Sorted (natural or Comparator)	Red-black tree	$O(\log n)$ operations; use when you need sorted iteration or range queries

Queue / Deque

```
Queue<Integer> q = new LinkedList<>();
q.offer(1); q.offer(2);
q.poll(); // removes and returns 1 (FIFO)

Deque<Integer> stack = new ArrayDeque<>(); // preferred Stack replacement
stack.push(1); stack.push(2);
stack.pop(); // removes and returns 2 (LIFO)

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5); pq.offer(1); pq.offer(3);
pq.poll(); // returns 1 - smallest first (min-heap by default)
```

Map implementations

	Ordering	Null keys/values	Thread-safe
HashMap	None	One null key, many null values	No
LinkedHashMap	Insertion order	Same as HashMap	No
TreeMap	Sorted by key	No null keys	No
Hashtable	None	No nulls at all	Yes (legacy, coarse-grained lock)
ConcurrentHashMap	None	No nulls	Yes (fine-grained, high-throughput)

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Asha", 30);
ages.put("Ravi", 25);
ages.merge("Asha", 1, Integer::sum);          // Asha → 31
for (Map.Entry<String, Integer> e : ages.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

HOW HASHMAP WORKS INTERNALLY

A `HashMap` stores entries in an array of buckets. `hashCode()` on the key picks a bucket; entries in the same bucket (a hash collision) form a linked list, or — since Java 8 — a red-black tree once a bucket exceeds 8 entries, keeping worst-case lookup $O(\log n)$ instead of $O(n)$. Overriding `equals()` without also overriding `hashCode()` (or vice versa) breaks `HashMap/HashSet` — always override both together.

RECRUITER PERSPECTIVE

"Which List/Map implementation would you use here, and why?" is one of the most common Java screening questions. The correct habit isn't memorizing Big-O — it's naming the actual access pattern (random reads? frequent inserts at the front? need sorted order? need thread safety?) and picking the structure that matches it.

Generics

Generics let a class or method operate on a type specified by the caller, catching type errors at compile time instead of runtime.

```
public class Box<T> {                                // generic class
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

Box<String> box = new Box<>();
box.set("hello");
// box.set(42);           // compile error – caught early, not at runtime

public static <T> T firstElement(List<T> list) {    // generic method
    return list.get(0);
}
```

Bounded types restrict what T can be; **wildcards** handle "unknown but related" types, governed by the mnemonic **PECS – Producer Extends, Consumer Super**:

```
<T extends Number> double sum(List<T> nums) { ... }    // bounded – T must be a Number

void copy(List<? extends Number> source, List<? super Number> dest) {
    // source PRODUCES values you read (extends) –
    // dest CONSUMES values you write (super)
}
```

Type erasure: generics exist only at compile time. The compiler checks types, then erases them, compiling `Box<String>` and `Box<Integer>` down to the same bytecode using `Object` internally with inserted casts. This is why you can't do `new T()` or check `instanceof T` at runtime.

INTERVIEW QUESTIONS – PART 4

1. Why is String immutable, and what problem does that solve?
2. Difference between StringBuilder and StringBuffer?
3. ArrayList vs LinkedList – when would you choose each?
4. How does HashMap resolve hash collisions internally?
5. Why must equals() and hashCode() be overridden together?
6. HashMap vs ConcurrentHashMap vs Hashtable – differences?
7. What is type erasure, and what limitation does it cause?
8. Explain PECS with an example.
9. When would you choose a TreeMap over a HashMap?
10. How does ArrayDeque replace both Stack and Queue's LinkedList usage?

PRACTICE NOW – 30 COLLECTIONS EXERCISES

- 1) Remove duplicates from a List using a Set. 2) Count word frequency using a HashMap. 3) Find the top-3 most frequent elements using a PriorityQueue. 4) Implement an LRU cache using LinkedHashMap. 5) Group a list of Strings by their length using a Map<Integer,List<String>>. 6) Reverse a LinkedList in place. 7) Detect a cycle in a linked list. 8) Merge two sorted lists into one sorted list. 9) Implement a stack-based bracket-matching validator. 10) Implement a queue using two stacks.
- 11) Find the intersection of two lists using a HashSet. 12) Sort a Map by its values. 13) Implement a simple in-memory cache with TTL using a Map. 14) Find the first non-repeating character in a string using a LinkedHashMap. 15) Flatten a nested list structure. 16) Implement a circular buffer using ArrayDeque. 17) Build a frequency-sorted word cloud from a paragraph. 18) Implement a simple graph adjacency list using Map<Node,List<Node>>. 19) Find all anagram groups in a list of words. 20) Implement a generic Pair<A,B> class.
- 21) Write a generic method to find the max of any Comparable list. 22) Implement a bounded generic Stack<T extends Comparable<T>> that always pops the max. 23) Build a generic Result<T,E> type representing success/failure. 24) Implement your own simplified HashMap using an array of buckets. 25) Simulate a job queue with PriorityQueue ordered by priority field. 26) Deduplicate a list while preserving order using LinkedHashSet. 27) Implement a sliding-window maximum using ArrayDeque. 28) Build a multi-map (one key → many values) using Map<K,List<V>>. 29) Find the median of a running stream of numbers using two heaps. 30) Implement Union-Find (Disjoint Set) with a Map-backed parent array.

Part 4 Quick Revision

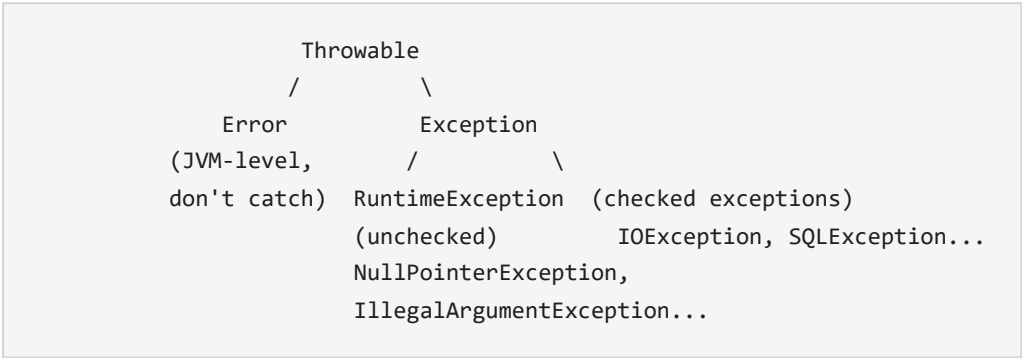
Need	Reach for
Fast random access, general list	ArrayList
Frequent head/tail insert, stack/queue	ArrayDeque
Unique elements, no order needed	HashSet
Unique elements, sorted	TreeSet
Key→value, fastest lookup	HashMap
Key→value, thread-safe	ConcurrentHashMap
Key→value, sorted by key	TreeMap

Next: Part 5 — Exception Handling: checked vs. unchecked, try-with-resources, and custom exceptions.

Part 5 – Exception Handling

Throwable hierarchy · try/catch/finally · custom exceptions · try-with-resources · Spring Boot error handling

The Throwable hierarchy



	Checked exceptions	Unchecked exceptions (RuntimeException)	Error
Must be declared/caught?	Yes – compiler enforces it	No	No
Represents	Recoverable, expected failure (file missing, network down)	Programming bugs (null deref, bad argument)	Serious JVM problems (OutOfMemoryError) – don't try to recover
Examples	IOException, SQLException	NullPointerException, IllegalArgumentException, IndexOutOfBoundsException	StackOverflowError, OutOfMemoryError

try / catch / finally / throw / throws

```
public double divide(int a, int b) {
    try {
        return a / b;
    } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero: " + e.getMessage());
        return 0;
    } finally {
        System.out.println("Division attempt finished");    // always runs
    }
}

public void readConfig() throws IOException {    // declares it may throw
    if (!configExists()) {
        throw new IOException("config.yml not found");    // throw = raise it
    }
}
```

Multi-catch handles several exception types identically without duplicating code:

```
try {
    process();
} catch (IOException | SQLException e) {
    log.error("Processing failed", e);
}
```

try-with-resources

Any class implementing `AutoCloseable` can be opened in a try-with-resources block; Java guarantees `close()` is called, even if an exception is thrown – no more forgotten `finally { conn.close(); }`.

```
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.executeUpdate();
}    // conn and stmt are both closed automatically here, in reverse order
```

Custom exceptions

```
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public void withdraw(double amount) {
    if (amount > balance) {
        throw new InsufficientFundsException(
            "Requested " + amount + " but balance is only " + balance);
    }
    balance -= amount;
}
```

PRO TIP

Prefer unchecked (extends `RuntimeException`) custom exceptions in Spring Boot services. Checked exceptions force every caller up the chain to declare or catch them – in a layered Controller→Service→Repository architecture that pollutes every method signature. Let unchecked exceptions propagate to a single `@ControllerAdvice` (Part 18) that converts them into clean HTTP error responses.

Exception propagation

```
void a() { b(); }
void b() { c(); }
void c() { throw new RuntimeException("boom"); }
// a() called b() called c(); the exception propagates back up
// through b() and a() until something catches it, or it reaches
// main() and crashes the program, printing a full stack trace.
```

DEBUG THIS

A stack trace reads top-to-bottom as "where it happened" → "who called that" → "who called that." The *first* line naming your own code (not a library class) is almost always where the real bug lives – start there, not at the top.

INTERVIEW QUESTIONS – PART 5

1. Checked vs. unchecked exceptions – give an example of each and explain when you'd use a custom one of each kind.
2. Does `finally` always run? What's the one case where it doesn't?
3. What problem does try-with-resources solve, and what interface must a resource implement?
4. Why is catching `Exception` (or worse, `Throwable`) broadly considered a bad practice?
5. How would you design a global exception-handling strategy for a REST API?
6. What's the difference between `throw` and `throws`?
7. Can a `finally` block override a return value from `try`? (Yes – explain why, and why you should avoid doing it.)

PRACTICE NOW – 15 EXERCISES

1) Write a custom `InvalidAgeException` and validate an age input. 2) Build a file reader using try-with-resources that reports a clean error if the file is missing. 3) Write a method that retries an operation up to 3 times on a specific exception before giving up. 4) Model a custom exception hierarchy: `ApplicationException` → `ValidationException`, `NotFoundException`, `ConflictException`. 5) Write a multi-catch block handling both `NumberFormatException` and `ArithmeticException` from a calculator. 6) Demonstrate exception chaining using the cause constructor (`new RuntimeException(msg, cause)`). 7) Write a method that wraps a checked `SQLException` into an unchecked custom exception. 8) Simulate a bank transfer that rolls back cleanly on `InsufficientFundsException`. 9) Build a simple validation framework that collects multiple field errors instead of throwing on the first one. 10) Write a resource class implementing `AutoCloseable` and prove `close()` runs even when the block throws.

Part 5 Quick Revision

Situation	Do this
External failure caller can recover from	Checked exception (rare in modern Spring code) or a Result type
Programming/validation error	Unchecked (RuntimeException subclass)
Managing a Closeable resource	try-with-resources, always
Handling errors across a whole REST API	One centralized @ControllerAdvice (Part 18)

Next: Part 6 — Java 8+ and Modern Java: lambdas, the Stream API, Optional, records, sealed classes, and pattern matching.

Part 6 – Java 8+ and Modern Java

Lambdas & functional interfaces · Stream API · Optional · var, records, sealed classes, pattern matching, text blocks

Lambda expressions & functional interfaces

A lambda is a compact, inline implementation of a functional interface (one abstract method). It replaces verbose anonymous classes.

```
// Before Java 8
Comparator<String> sc-camel-by-length = new Comparator<String>() {
    public int compare(String a, String b) { return a.length() - b.length(); }
};

// With a lambda
Comparator<String> sc-camel-by-length = (a, b) -> a.length() - b.length();

// Method reference – even shorter when a lambda just calls one existing method
List<String> names = List.of("Ravi", "Asha", "Om");
names.forEach(System.out::println);
```

Interface	Method	Meaning
Predicate<T>	boolean test(T t)	A yes/no check
Consumer<T>	void accept(T t)	Do something with a value, return nothing
Supplier<T>	T get()	Produce a value with no input
Function<T,R>	R apply(T t)	Transform T into R
BiFunction<T,U,R>	R apply(T t, U u)	Combine two inputs into one output

The Stream API

A **Stream** is a pipeline for processing sequences of data declaratively – "what," not "how."
Streams are lazy (nothing runs until a terminal operation), and each intermediate step returns a new stream.

```
List<String> names = List.of("Ravi", "Asha", "Om", "Priya", "Amit");

List<String> result = names.stream()
    .filter(n -> n.length() > 2)      // keep names longer than 2 chars
    .map(String::toUpperCase)         // transform each
    .sorted()                         // sort alphabetically
    .collect(Collectors.toList());    // terminal operation – materializes the result
// [AMIT, ASHA, PRIYA, RAVI]

long count = names.stream().filter(n -> n.startsWith("A")).count();

int sc-camel-total-length = names.stream()
    .mapToInt(String::length)
    .sum();                          // reduce-style terminal operation

Map<Integer, List<String>> sc-camel-by-length = names.stream()
    .collect(Collectors.groupingBy(String::length));
// {2=[Om], 4=[Ravi, Asha, Amit], 5=[Priya]}

Map<Boolean, List<String>> partitioned = names.stream()
    .collect(Collectors.partitioningBy(n -> n.length() > 3));

List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));
List<Integer> flat = nested.stream()
    .flatMap(List::stream)            // flattens List<List<Integer>> into List<Integer>
    .collect(Collectors.toList());    // [1, 2, 3, 4]

int sc-camel-sum-of-all = List.of(1,2,3,4,5).stream()
    .reduce(0, Integer::sum);         // fold to a single value, starting from 0
```

Operation	Kind	Purpose
filter, map, sorted, distinct, limit, skip, flatMap	Intermediate (lazy)	Build the pipeline; return a new Stream
collect, reduce, count, forEach, sum, anyMatch	Terminal (eager)	Trigger execution and produce a result

COMMON MISTAKE

Streams can only be consumed once — calling a terminal operation twice on the same stream throws `IllegalStateException`. If you need the data twice, collect it to a `List` first, then stream the `List` again.

Optional

`Optional<T>` is a container that may or may not hold a value — designed to make the possibility of "no value" explicit in a method's return type, instead of silently returning `null`.

```
Optional<User> user = userRepository.findById(id);

user.ifPresent(u -> System.out.println(u.getName()));

String name = user.map(User::getName).orElse("Unknown");

User found = user.orElseThrow(() -> new UserNotFoundException(id));
```

PRO TIP

Use `Optional` only as a *return type* from methods that might have nothing to return — never as a field type, method parameter, or inside a collection. Calling `.get()` without checking `.isPresent()` first defeats its entire purpose.

Modern Java language features

```
// var – local type inference (Java 10+); the type is still fixed at compile time
var list = new ArrayList<String>();    // inferred as ArrayList<String>

// Records – concise immutable data carriers (Java 16+)
public record Money(String currency, long cents) {}

// Sealed classes – restrict which classes may extend/implement (Java 17+)
public sealed interface Shape permits Circle, Square {}
public record Circle(double radius) implements Shape {}
public record Square(double side) implements Shape {}

// Pattern matching + switch expressions (Java 21) – exhaustive, no default needed
double area(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square s -> s.side() * s.side();
    };
}

// Text blocks (Java 15+) – multi-line strings without escaping
String json = """
{
    "name": "Asha",
    "role": "Backend Developer"
}
""";

// Modern date/time API (java.time, Java 8+) – replaces the old, mutable Date/Calendar
LocalDate today = LocalDate.now();
LocalDateTime meeting = LocalDateTime.of(2026, 9, 1, 14, 30);
Duration sc-camel-time-left = Duration.between(LocalDateTime.now(), meeting);
```

REAL-WORLD INSIGHT

Sealed interfaces + records + pattern-matching switch give Java algebraic-data-type modeling similar to Kotlin/Scala: the compiler *guarantees* every case is handled at compile time. This combination is increasingly used to model API result types (e.g. sealed interface `ApiResponse` permits `Success`, `Failure`) in modern Spring Boot codebases.

INTERVIEW QUESTIONS – PART 6

1. What is a functional interface, and why is `Comparator` one?
2. Difference between `map` and `flatMap`?
3. Why are streams lazy, and what triggers execution?
4. Why can't a stream be reused after a terminal operation?
5. What problem does `Optional` solve, and what's the anti-pattern to avoid with it?
6. `groupingBy` vs. `partitioningBy` – when would you use each?
7. What guarantee do sealed classes give you when combined with a pattern-matching switch?
8. Why is `java.time` preferred over the legacy `Date/Calendar` classes?
9. Is `var` dynamic typing? Explain why not.

PRACTICE NOW – 40 STREAM API EXERCISES

1) Filter even numbers from a list. 2) Square every number in a list. 3) Find the sum of squares of even numbers. 4) Find the max/min of a list using streams. 5) Count strings longer than 5 characters. 6) Sort a list of employees by salary descending. 7) Group employees by department. 8) Find the average salary per department. 9) Get a comma-separated string of all names using joining(). 10) Remove duplicates from a list of objects by a field, using a Set-backed filter.

11) Find the second-highest salary using sorted + skip. 12) Convert a List<String> to a Map<String,Integer> of string→length. 13) Partition numbers into even/odd. 14) Flatten a List<List<String>>. 15) Find all unique characters across a list of words. 16) Count occurrences of each word in a sentence using groupingBy + counting(). 17) Find the longest string in a list. 18) Convert a list of Persons to a list of names only. 19) Check if all elements match a condition (allMatch). 20) Check if any element matches a condition (anyMatch).

21) Sum all order totals using mapToDouble + sum. 22) Find employees earning above the average salary. 23) Sort a list by two fields (last name, then first name) using thenComparing. 24) Build a Map<Department,List<Employee>> sorted by employee count. 25) Find duplicate elements in a list using groupingBy + counting + filter. 26) Generate the first 10 Fibonacci numbers using Stream.iterate. 27) Generate 5 random numbers using Stream.generate. 28) Convert an array to a stream and back. 29) Find the top-3 highest-paid employees using sorted + limit. 30) Compute a running total (prefix sum) using reduce.

31) Collect results into a TreeMap for sorted keys. 32) Use Collectors.toMap to build a lookup table, handling merge conflicts. 33) Chain multiple filters into one predicate using Predicate.and(). 34) Use IntStream.rangeClosed to sum 1-100. 35) Find distinct first letters used across a list of names. 36) Build a CSV-like summary line per group using joining with a prefix/suffix. 37) Convert nested employee-department data into Map<Department, Double> of average salary. 38) Use Optional chained with map/orElse to safely extract a nested field. 39) Parallelize a large sum computation with parallelStream and compare timing conceptually. 40) Build a Collectors.summarizingInt report (min/max/avg/sum/count) for a list of prices.

Part 6 Quick Revision

Feature	One-line takeaway
Lambda	Inline implementation of a one-method (functional) interface
Stream	Lazy pipeline: intermediate ops build it, one terminal op runs it
Optional	Explicit "may be absent" return type – not for fields/params
Records	Immutable data carriers with generated equals/hashCode/toString
Sealed + switch pattern matching	Compiler-enforced exhaustive handling of a closed type hierarchy

END OF INSTALLMENT 1 – PARTS 1-6

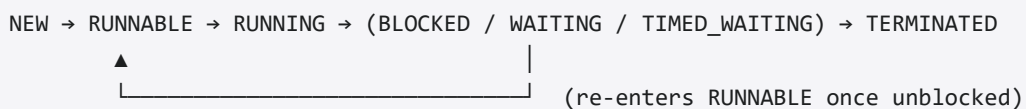
This covers the complete Java language foundation: programming basics, core syntax, OOP, collections, exceptions, and modern Java. Part 7 (Multithreading & Concurrency) through Part 45 (Cheat Sheets) continue in the next installments, building toward Spring, Spring Boot, databases, security, microservices, system design, DSA, and full interview preparation – see the [Master Index](#) for the full roadmap.

Part 7 – Multithreading & Concurrency

Threads · Executors · Synchronization · Locks · Atomics · Concurrent collections ·
CompletableFuture · Virtual threads

Process vs. thread, and the thread lifecycle

A **process** is an independent running program with its own memory space. A **thread** is a lightweight unit of execution inside a process; all threads in one process share the same heap memory, which is what makes concurrency both powerful and dangerous.



```
// Two ways to define work for a thread
Runnable task = () -> System.out.println("Running on " +
Thread.currentThread().getName());
new Thread(task).start();

Callable<Integer> sc-camel-with-result = () -> 42; // Callable can return a value and
throw checked exceptions
```

ExecutorService & thread pools

Creating a raw Thread per task doesn't scale — thread creation is expensive and unbounded threads can exhaust memory. An **ExecutorService** manages a reusable pool of worker threads.

```
ExecutorService pool = Executors.newFixedThreadPool(4);

Future<Integer> future = pool.submit(() -> {
    Thread.sleep(100);
    return 42;
});

Integer result = future.get(); // blocks until the task completes
pool.shutdown(); // always shut down when finished
```

Factory method	Pool behavior	Use when
<code>newFixedThreadPool(n)</code>	Fixed number of reusable threads	Predictable, CPU-bound workloads
<code>newCachedThreadPool()</code>	Grows/shrinks as needed	Many short-lived tasks
<code>newSingleThreadExecutor()</code>	One worker thread, serialized tasks	Tasks that must run in order, one at a time
<code>newVirtualThreadPerTaskExecutor()</code>	One cheap virtual thread per task (Java 21+)	Massive numbers of blocking I/O-bound tasks

Synchronization

When multiple threads read and write shared state without coordination, you get a **race condition** – the outcome depends on unpredictable timing.

```
class Counter {
    private int count = 0;
    public synchronized void increment() {    // only one thread at a time may run this
        count++;                             // count++ is NOT atomic: read, add, write
    }
}
```

Mechanism	What it guarantees
<code>synchronized</code>	Mutual exclusion (one thread at a time) + visibility of changes across threads
<code>volatile</code>	Visibility only – every read sees the latest write, but doesn't make compound operations atomic
<code>ReentrantLock</code>	Like <code>synchronized</code> , but supports <code>tryLock()</code> , timeouts, and interruptible waits
<code>ReadWriteLock</code>	Many concurrent readers OR one exclusive writer – good for read-heavy shared data
Atomic classes (<code>AtomicInteger</code> , etc.)	Lock-free, hardware-level atomic operations for a single variable

```

AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet();    // thread-safe, no explicit lock needed

private final ReentrantLock lock = new ReentrantLock();
public void transfer() {
    lock.lock();
    try {
        // critical section
    } finally {
        lock.unlock();    // always unlock in finally
    }
}

```

Deadlock, starvation, livelock

Problem	What happens
Deadlock	Two+ threads each hold a lock the other needs – both wait forever
Starvation	A thread never gets CPU time or a lock because others are prioritized ahead of it
Livelock	Threads keep changing state in response to each other but make no real progress

PRO TIP

The classic deadlock fix: always acquire multiple locks in the same global order across every thread. If Thread A locks (accountX, then accountY) and Thread B locks (accountY, then accountX), you have a deadlock waiting to happen. Sort by a stable id before locking.

CompletableFuture & parallel streams

```

CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> fetchUserFromApi())
    .thenApply(user -> user.getName().toUpperCase())
    .exceptionally(ex -> "Unknown");

List<Integer> nums = List.of(1,2,3,4,5);
int sum = nums.parallelStream().mapToInt(Integer::intValue).sum();    // splits work across
threads

```

Virtual threads (Java 21)

Virtual threads are lightweight threads managed by the JVM rather than the OS – you can spin up millions of them. They make blocking I/O code (a JDBC call, an HTTP call) scale like reactive code, without rewriting it in a reactive style.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> callSlowService()); // each gets its own virtual thread – cheap
    }
}
```

INTERVIEW QUESTIONS – PART 7

1. Difference between synchronized and volatile?
2. Why is `count++` not thread-safe even on a simple `int`?
3. What causes a deadlock, and how do you prevent it?
4. `ExecutorService` vs. raw `Thread` – why prefer the former?
5. What problem do virtual threads solve compared to platform threads?
6. How does `ConcurrentHashMap` achieve thread safety without locking the whole map?
7. Explain the difference between `Runnable` and `Callable`.
8. What does `CompletableFuture` add over a plain `Future`?

MINI CHALLENGE

Build a thread-safe bounded bank account: `deposit()/withdraw()` using `ReentrantLock`, a `transferBetween(a, b)` method that avoids deadlock by lock-ordering, and a stress test that spawns 100 threads hammering the same account to prove no money is lost or created.

Part 7 Quick Revision

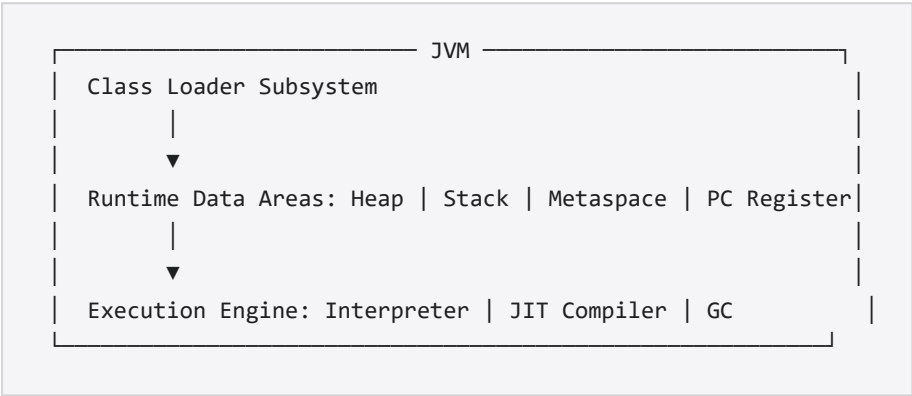
Need	Reach for
Simple exclusive section	synchronized
Timeout / tryLock / interruptible	ReentrantLock
Single counter/flag	Atomic classes
Thread-safe map	ConcurrentHashMap
Many blocking I/O tasks	Virtual threads

Next: Part 8 — JVM Internals: class loading, memory areas, garbage collection, and JIT compilation.

Part 8 – JVM Internals

Class loading · Runtime memory areas · Garbage collection · JIT compilation · Tuning & profiling basics

JVM architecture overview



Class loading

Classes are loaded lazily, on first use, in three phases: **Loading** (reading the .class file into memory), **Linking** (verifying bytecode, allocating memory for static fields, resolving references), and **Initialization** (running static initializers and static field assignments). A hierarchy of class loaders (Bootstrap → Platform → Application) delegates upward first, so core Java classes can never be shadowed by application code.

Runtime data areas

Area	Stores	Shared across threads?
Heap	All objects and arrays (via new)	Yes
Stack	Method call frames — local variables, partial results	No — one per thread
Metaspace	Class metadata (replaced PermGen since Java 8)	Yes
PC Register	Address of the current instruction per thread	No — one per thread

The heap is further divided generationally: **Young Generation** (Eden + two Survivor spaces, where new objects are born and most die quickly) and **Old Generation** (long-lived objects promoted after surviving several collections). This generational split exists because most objects die young — collecting Young Gen frequently and cheaply is far more efficient than scanning the whole heap every time.

Garbage collection

Collector	Strategy	Best for
Serial GC	Single-threaded, stop-the-world	Small heaps, single-core environments
Parallel GC	Multi-threaded, stop-the-world	Throughput-focused batch workloads
G1 (Garbage First)	Region-based, mostly concurrent, balances throughput and pause time	Default for most modern server apps
ZGC	Fully concurrent, sub-millisecond pauses even on huge heaps	Very large heaps, strict latency requirements

```
# Common JVM tuning flags
java -Xms512m -Xmx2g -XX:+UseG1GC -jar app.jar
# -Xms: initial heap -Xmx: max heap -XX:+UseG1GC: pick the G1 collector
```

JIT compilation

The JVM starts by interpreting bytecode (slow but instant startup). The **JIT (Just-In-Time) compiler** monitors which methods run frequently ("hot" methods) and compiles them directly to native machine code at runtime, so repeated execution gets much faster — this is why long-running Java processes often perform better the longer they run ("warm-up").

Common memory errors

Error	Cause
StackOverflowError	Uncontrolled recursion — stack frames pile up past the thread's stack size
OutOfMemoryError: Java heap space	Heap exhausted — often a memory leak (objects held onto via a growing static collection, unclosed resources, listeners never unregistered)
OutOfMemoryError: Metaspace	Too many classes loaded, often from dynamic class generation/classloader leaks

DEBUG THIS

A slow memory leak in production usually shows as a heap-usage graph that saws upward after every GC instead of returning to baseline. Take a heap dump (`jmap -dump`) at two points in time and diff the retained objects in a profiler (VisualVM, Eclipse MAT) — the class with the fastest-growing instance count is almost always your leak.

INTERVIEW QUESTIONS — PART 8

1. Walk through what happens from `java -jar app.jar` to your `main()` method running.
2. Why is the heap divided into Young and Old generations?
3. What replaced PermGen in modern Java, and why?
4. `StackOverflowError` vs. `OutOfMemoryError` — different causes?
5. What does the JIT compiler do that the interpreter alone can't achieve efficiently?
6. When would you choose G1 vs. ZGC?

Part 8 Quick Revision

Term	One-liner
Heap	Shared object storage, garbage-collected
Stack	Per-thread call frames, not garbage-collected — unwound on return
Metaspace	Class metadata, native memory (off-heap)
G1 GC	Default modern collector, balances pause time and throughput

Next: Part 9 – Files, I/O & Serialization: streams, NIO, and JSON with Jackson.

Part 9 – Files, I/O & Serialization

Streams · Readers/Writers · Buffered I/O · NIO Paths/Files · JSON with Jackson

Streams: byte vs. character

Family	Base classes	Handles
Byte streams	InputStream / OutputStream	Raw bytes – images, binary files, network sockets
Character streams	Reader / Writer	Text, with charset decoding/encoding handled for you

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} // auto-closed

try (BufferedWriter writer = new BufferedWriter(new FileWriter("out.txt"))) {
    writer.write("Hello, file!");
}
```

Buffered streams wrap raw streams to batch reads/writes in memory, drastically reducing expensive I/O system calls – always prefer `BufferedReader/BufferedWriter` over raw `FileReader/FileWriter` for anything beyond trivial use.

NIO – Path and Files

The modern `java.nio.file` API is preferred over the legacy `java.io.File` for new code.

```
Path path = Paths.get("data.txt");
List<String> sc-camel-all-lines = Files.readAllLines(path);
Files.writeString(path, "new content", StandardOpenOption.APPEND);
boolean exists = Files.exists(path);
Files.walk(Paths.get("src")).filter(Files::isRegularFile).forEach(System.out::println);
```

Serialization & JSON with Jackson

Java's built-in `Serializable` (binary, JVM-specific) is rarely used in modern backend work. Instead, Spring Boot uses **Jackson** to convert Java objects to/from JSON automatically at the REST boundary.

```
public record UserDto(String name, int age) {}

ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(new UserDto("Asha", 30));
// {"name":"Asha","age":30}

UserDto parsed = mapper.readValue(json, UserDto.class);
```

In a Spring Boot `@RestController`, this happens transparently: a method returning a DTO is serialized to JSON by Jackson automatically, and a `@RequestBody` parameter is deserialized from the incoming JSON the same way — no manual `ObjectMapper` calls needed (Part 18 covers this in full).

COMMON MISTAKE

Jackson needs either a no-args constructor + setters, or (for records/final fields) a way to see all the constructor parameter names — usually satisfied automatically for records and Lombok-generated classes, but a hand-written immutable class without annotations can fail to deserialize with a confusing `InvalidDefinitionException`.

INTERVIEW QUESTIONS – PART 9

1. Byte streams vs. character streams — when do you need each?
2. Why wrap a `FileReader` in a `BufferedReader`?
3. NIO's `Files` vs. legacy `File` — what's the practical benefit?
4. How does Jackson convert a Java object to JSON without you calling it explicitly in a Spring Boot controller?
5. What is `try-with-resources` solving here specifically for file streams?

PRACTICE NOW

1) Write a program that counts lines/words/characters in a text file. 2) Copy a file using NIO's Files.copy. 3) Recursively list all .java files in a directory tree. 4) Read a CSV file and parse it into a List of records. 5) Serialize a List of objects to a JSON file and read it back with Jackson.

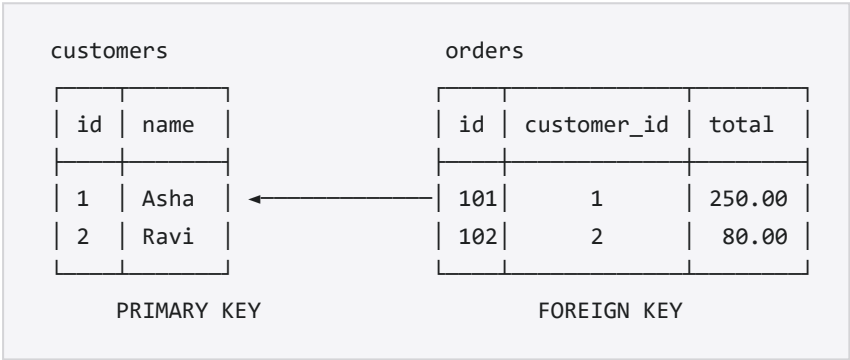
Next: Part 10 – Database Fundamentals: relational modeling, keys, normalization, and transactions.

Part 10 – Database Fundamentals

Relational databases · Keys & constraints · Normalization · Indexes · Transactions & ACID · Joins

What is a relational database?

A relational database organizes data into **tables** (rows and columns), where relationships between tables are expressed through shared key values rather than nesting data inside itself. This course uses **PostgreSQL** as the reference database throughout.



Keys & constraints

Concept	Meaning
Primary key	Uniquely identifies each row; cannot be null or duplicated
Foreign key	A column referencing another table's primary key, enforcing referential integrity
NOT NULL	The column must always have a value
UNIQUE	No two rows may share this value
CHECK	A row-level rule (e.g. <code>price > 0</code>) enforced by the database itself

Normalization

Form	Rule
1NF	Every column holds a single, atomic value – no repeating groups or lists in a cell
2NF	1NF + every non-key column depends on the whole primary key, not part of it
3NF	2NF + no non-key column depends on another non-key column (no transitive dependency)

Normalization eliminates data duplication and update anomalies, at the cost of needing joins to reassemble related data. Real systems sometimes deliberately **denormalize** (duplicate some data) to optimize read-heavy paths – a tradeoff, not a rule violation.

Transactions & ACID

Property	Guarantee
Atomicity	A transaction's operations all succeed, or all roll back – no partial state
Consistency	A transaction moves the database from one valid state to another, respecting all constraints
Isolation	Concurrent transactions don't see each other's uncommitted changes
Durability	Once committed, data survives even a crash immediately after

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;  -- both updates succeed together, or neither does
```

Indexes

An index is a separate, sorted data structure (usually a B-tree) that lets the database find rows by a column's value without scanning the whole table – trading extra storage and slower writes for much faster reads on that column.

PRO TIP

Index every foreign key column and every column you frequently filter or sort by (**WHERE/ORDER BY**). Don't over-index – every extra index slows down every **INSERT/UPDATE** on that table.

INTERVIEW QUESTIONS – PART 10

1. Explain ACID with a real transaction example.
2. What is the difference between a primary key and a unique constraint?
3. Why would you deliberately denormalize a schema?
4. How does an index speed up a query, and what's the cost?
5. What is referential integrity, and how do foreign keys enforce it?

Next: Part 11 – Complete SQL for Java Developers: SELECT through window functions, with 250+ practice queries.

Part 12 – JDBC

Connections · Statements · PreparedStatement · Transactions · Connection pooling · The DAO pattern

JDBC architecture

Java App → JDBC API → JDBC Driver (vendor-specific) → Database

JDBC (Java Database Connectivity) is the standard API for talking to any relational database; a vendor-supplied **driver** (e.g. the PostgreSQL JDBC driver) implements it. Spring Data JPA (Part 19) is built on top of JDBC – understanding this layer explains what JPA is abstracting away.

Connecting and querying

```
String url = "jdbc:postgresql://localhost:5432/mydb";
try (Connection conn = DriverManager.getConnection(url, "user", "pass");
    PreparedStatement stmt = conn.prepareStatement(
        "SELECT id, name FROM employees WHERE department = ?")) {

    stmt.setString(1, "Engineering");
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        }
    }
}
```

PRODUCTION NOTE – SQL INJECTION

Never build SQL with string concatenation of user input ("...WHERE name = '" + userInput + "'") – it allows SQL injection. **PreparedStatement** with ? placeholders sends the query and data separately, making injection structurally impossible. Always use PreparedStatement for any query containing external input.

Transactions & batch processing

```
conn.setAutoCommit(false);
try {
    stmt1.executeUpdate();
    stmt2.executeUpdate();
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}

PreparedStatement batch = conn.prepareStatement("INSERT INTO logs(msg) VALUES (?)");
for (String msg : messages) {
    batch.setString(1, msg);
    batch.addBatch();
}
batch.executeBatch(); // sends all inserts in one round trip – much faster than one-by-one
```

Connection pooling

Opening a raw database connection is expensive (TCP handshake, authentication). A **connection pool** (HikariCP is Spring Boot's default) keeps a set of live connections ready to reuse, dramatically cutting per-request latency.

The DAO pattern

```
public interface EmployeeDao {
    Optional<Employee> findById(int id);
    List<Employee> findAll();
    void save(Employee e);
}

public class JdbcEmployeeDao implements EmployeeDao {
    private final DataSource dataSource;
    public JdbcEmployeeDao(DataSource dataSource) { this.dataSource = dataSource; }

    public Optional<Employee> findById(int id) {
        String sql = "SELECT id, name FROM employees WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            try (ResultSet rs = stmt.executeQuery()) {
                return rs.next()
                    ? Optional.of(new Employee(rs.getInt("id"), rs.getString("name")))
                    : Optional.empty();
            }
        } catch (SQLException e) {
            throw new RuntimeException("Query failed", e);
        }
    }
    // findAll(), save() follow the same shape
}
```

The **Data Access Object** pattern isolates all raw SQL behind an interface, so the rest of the application depends only on that contract – exactly what `JpaRepository` gives you for free in Part 19.

INTERVIEW QUESTIONS – PART 12

1. Statement vs. PreparedStatement – why does the latter prevent SQL injection?
2. Why use connection pooling instead of opening a new connection per request?
3. What does `setAutoCommit(false)` change, and why would you use it?
4. What problem does the DAO pattern solve architecturally?
5. How does batch execution improve performance over looped single inserts?

Next: Part 13 – Web Development Fundamentals: HTTP, REST, and the full client-to-database request lifecycle.

Part 13 – Web Development Fundamentals

HTTP/HTTPS · DNS · REST · JSON · Status codes · Headers, cookies & sessions · Auth basics

The full lifecycle

```
Browser → DNS lookup → TCP connection → TLS handshake (HTTPS)
  → HTTP Request → Server (Spring Boot) → Application logic → Database
  → HTTP Response → Browser renders result
```

DNS translates a domain name into an IP address. **TCP/IP** establishes a reliable connection between client and server over that address. **HTTPS** is HTTP layered over TLS encryption – the browser and server perform a handshake to agree on encryption keys before any data is exchanged.

HTTP methods & status codes

Method	Purpose	Idempotent?
GET	Read a resource	Yes
POST	Create a resource	No
PUT	Replace a resource entirely	Yes
PATCH	Partially update a resource	No (generally)
DELETE	Remove a resource	Yes

Code range	Meaning	Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict
5xx	Server error	500 Internal Server Error, 503 Service Unavailable

REST principles

REST (Representational State Transfer) models resources as URLs and uses HTTP methods as verbs on them. A resource is represented as data (usually JSON) rather than as an action.

GET	/api/orders	→ list all orders
GET	/api/orders/42	→ get order 42
POST	/api/orders	→ create an order
PUT	/api/orders/42	→ replace order 42
DELETE	/api/orders/42	→ delete order 42

COMMON MISTAKE

GET /api/deleteOrder?id=42 is not REST – it puts a verb in the URL and uses GET to mutate state, which also breaks caching and can be triggered accidentally by prefetching. The resource-plus-HTTP-verb pattern above is the correct shape.

Headers, cookies, sessions, auth

Concept	Purpose
Header	Metadata about the request/response (Content-Type, Authorization, Cache-Control)
Cookie	Small piece of data the server asks the browser to store and resend on future requests
Session	Server-side state tied to a client, usually identified by a session-id cookie
Authentication	Confirming who the caller is
Authorization	Deciding what that caller is allowed to do

Modern Spring Boot APIs typically favor stateless, token-based authentication (JWT – Part 20) over server-side sessions, since it scales horizontally without needing shared session storage across servers.

INTERVIEW QUESTIONS – PART 13

1. What makes an HTTP method "idempotent," and why does it matter for retries?
2. Difference between authentication and authorization?
3. 401 vs. 403 – what's the practical difference in when you'd return each?
4. What is a session, and why do modern REST APIs often avoid them in favor of tokens?
5. Why is HTTPS necessary even for "non-sensitive" data?
6. Explain what happens during a TLS handshake, at a high level.

Next: Part 14 – Maven & Gradle: build tools, dependency management, and project structure.

Part 14 – Maven & Gradle

pom.xml · Dependencies & scopes · Build lifecycle · Profiles · Multi-module projects · Gradle basics

Why a build tool?

A build tool automates compiling code, managing external libraries (dependencies), running tests, and packaging the application into a deployable artifact (a `.jar`). Maven is the most common choice for Spring Boot projects and is used as the primary tool throughout this course.

pom.xml

```
<project>
  <groupId>com.example</groupId>
  <artifactId>order-service</artifactId>
  <version>1.0.0</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
  </parent>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

The `spring-boot-starter-parent` pins compatible dependency versions for the whole Spring ecosystem, so you rarely specify a version yourself — this is **dependency management** at work, and it's exactly what prevents version-conflict headaches.

Dependency scope

Scope	Available during
compile (default)	Compilation, testing, runtime — packaged into the final jar
test	Test compilation and execution only
provided	Compilation only — expected to be supplied by the runtime environment
runtime	Execution and testing, not compilation (e.g. a JDBC driver)

Build lifecycle

```
mvn validate    # checks project structure
mvn compile     # compiles source
mvn test        # runs unit tests
mvn package     # produces the .jar/.war
mvn install     # installs the artifact to the local repository
mvn verify      # runs integration checks
```

Each phase runs every phase before it — `mvn package` implicitly compiles and tests first.

Profiles & multi-module projects

Profiles let you swap configuration per environment (dev vs. prod dependency sets, for instance). A **multi-module** Maven project uses a parent `pom.xml` that lists child modules — common for splitting a large system into independently buildable pieces (e.g. `api-gateway`, `order-service`, `common-lib`) while sharing dependency management.

Gradle, briefly

```
// build.gradle.kts (Kotlin DSL)
plugins { id("org.springframework.boot") version "3.3.0" }
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

Gradle uses a programmable DSL (Groovy or Kotlin) instead of Maven's declarative XML, and builds incrementally with a dependency graph, making large multi-module builds noticeably faster. Either tool is a fine choice for Spring Boot – this course's project examples use Maven for consistency.

INTERVIEW QUESTIONS – PART 14

1. What does spring-boot-starter-parent actually solve for you?
2. compile vs. provided vs. runtime scope – give an example dependency for each.
3. What's the practical benefit of a multi-module Maven project?
4. Maven vs. Gradle – what's the core structural difference?
5. How would you resolve two dependencies pulling in conflicting versions of the same library?

Next: Part 15 – Git & GitHub for Java Developers.

Part 15 – Git & GitHub for Java Developers

Commits & branches · Merge vs. rebase · Pull requests · GitHub Actions · Conventional commits

Core concepts

Term	Meaning
Repository	A project's full version history
Commit	A saved snapshot of changes with a message
Branch	An independent line of development
Merge	Combining one branch's history into another
Pull request	A request to merge a branch, opened for review on GitHub

```
git init
git add .
git commit -m "feat: add order creation endpoint"
git branch feature/order-api
git checkout feature/order-api
git push origin feature/order-api
```

Merge vs. rebase

```
git checkout main
git merge feature/order-api      # creates a merge commit, preserves both histories

git checkout feature/order-api
git rebase main                  # replays your commits on top of main – linear history
```

	Merge	Rebase
History	Preserves branch structure, extra merge commit	Linear, cleaner log
Safety	Never rewrites history	Rewrites commit hashes – never rebase a shared/public branch
Use for	Merging a finished feature into main	Cleaning up your own branch before opening a PR

Resolving conflicts

```
<<<<<< HEAD
int sc-camel-max-retries = 3;
=====
int sc-camel-max-retries = 5;
>>>>>> feature/order-api
```

Git marks the conflicting region in both versions; you manually edit to the correct final content, remove the markers, then `git add` the file and continue the merge/rebase.

Conventional commits & .gitignore

```
feat: add JWT authentication filter
fix: correct null pointer in OrderService.calculateTotal
docs: update README with Docker instructions
refactor: extract validation logic into a separate class
test: add unit tests for PaymentService
```

```
# .gitignore for a Spring Boot project
target/
*.class
.idea/
*.iml
application-local.yml
.env
```

GitHub Actions (CI basics)

```
# .github/workflows/build.yml
name: Build and Test
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: mvn -B test
```

This pipeline runs automatically on every push and PR, catching failures before a human ever reviews the code. Part 27 builds this out into a full CI/CD deployment pipeline.

INTERVIEW QUESTIONS – PART 15

1. Merge vs. rebase – when would you use each, and why is rebasing a shared branch dangerous?
2. What's in a typical .gitignore for a Spring Boot project, and why exclude application-local.yml?
3. Walk through resolving a merge conflict in a Java file.
4. What is a pull request, and what does it add over pushing straight to main?
5. What's the value of conventional commit messages for a team?

Next: Part 16 – Spring Framework from Zero: IoC, dependency injection, and beans.

Part 16 – Spring Framework from Zero

IoC & dependency injection · ApplicationContext · Beans & their lifecycle · Stereotypes · Scopes & profiles

The problem Spring solves

Without a framework, every class that needs a collaborator must construct it directly – `new MySQLOrderRepository()` baked right into `OrderService`. This tightly couples code, makes swapping implementations (e.g. for testing) painful, and scatters object-creation logic everywhere. Spring's **IoC container** flips this: instead of your objects creating their own dependencies, the *container* creates and wires them together – "Inversion of Control."

Dependency Injection

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;    // dependency

    @Autowired                                     // constructor injection –
    preferred
    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
}

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {}
```

Spring scans for classes annotated `@Component` (and its specializations), creates one instance of each (a **bean**), and injects them wherever they're needed – you never call `new` on your own services.

Injection style	How	Recommended?
Constructor injection	Dependencies passed via constructor, fields can be <code>final</code>	Yes – always prefer this
Field injection	<code>@Autowired</code> directly on a field	No – hides dependencies, breaks immutability, harder to unit test without Spring
Setter injection	<code>@Autowired</code> on a setter method	Rare – only for optional dependencies

PRO TIP

With one constructor, Spring Boot 3+ doesn't even require `@Autowired` – it's implicit. Constructor injection also means you literally cannot construct the class without its dependencies, catching missing-wiring mistakes at compile time instead of a runtime `NullPointerException`.

Stereotype annotations

Annotation	Marks
<code>@Component</code>	A generic Spring-managed bean
<code>@Service</code>	A business-logic class (semantically a <code>@Component</code>)
<code>@Repository</code>	A data-access class – also enables automatic exception translation
<code>@Controller</code> / <code>@RestController</code>	A web layer class handling HTTP requests
<code>@Configuration</code>	A class that declares beans via <code>@Bean</code> methods

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();    // manually configured bean for a 3rd-party class
    }
}
```

Bean lifecycle & the ApplicationContext

Instantiate → Populate properties (DI) → @PostConstruct →
Bean is ready for use → @PreDestroy → Destroyed

The **ApplicationContext** is the running IoC container: it performs component scanning, builds every bean, wires dependencies, and manages each bean's lifecycle from startup to shutdown.

Bean scopes & profiles

Scope	Instances
singleton (default)	One shared instance per container
prototype	A new instance every time it's requested
request / session	Web-aware: one instance per HTTP request/session

```
@Profile("dev")
@Bean
public DataSource devDataSource() { return new H2DataSource(); }

@Profile("prod")
@Bean
public DataSource prodDataSource() { return new PostgresDataSource(); }
```

Profiles let one codebase behave differently per environment – activated via `spring.profiles.active=dev` in configuration or an environment variable.

INTERVIEW QUESTIONS – PART 16

1. What problem does Inversion of Control actually solve?
2. Why is constructor injection preferred over field injection?
3. What is a Spring bean, and what creates it?
4. @Component vs. @Service vs. @Repository – functionally, what's different about @Repository?
5. Singleton vs. prototype bean scope – give a scenario for prototype.
6. What is the ApplicationContext responsible for, concretely?
7. How do Spring profiles let the same code run differently in dev vs. prod?

Next: Part 17 – Spring Boot: auto-configuration, starters, and building your first application.

Part 17 — Spring Boot

Starters · Auto-configuration · Embedded servers · Configuration files · Profiles · Actuator

Spring vs. Spring Boot

Spring Framework gives you IoC, DI, and the building blocks; historically, wiring a Spring web app meant hand-writing pages of XML/Java configuration. **Spring Boot** is "opinionated Spring" — it auto-configures sensible defaults, embeds a web server, and gets you from zero to a running application in minutes.

Starters & auto-configuration

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A **starter** is a curated dependency bundle: `spring-boot-starter-web` pulls in Spring MVC, an embedded Tomcat server, and Jackson together — one line instead of a dozen. **Auto-configuration** then inspects what's on the classpath and configures matching beans automatically: seeing Tomcat on the classpath, Boot configures and starts an embedded web server without you writing a line of server setup code.

HOW IT WORKS INTERNALLY

`@SpringBootApplication` bundles three annotations: `@Configuration`, `@ComponentScan`, and `@EnableAutoConfiguration`. The last one reads a list of candidate auto-configuration classes (each guarded by `@ConditionalOnClass`/`@ConditionalOnMissingBean` checks) and applies only the ones whose conditions match your classpath and existing beans — this is why adding a starter "just works" and why defining your own bean of the same type silently overrides Boot's default.

Your first Spring Boot application

```
@SpringBootApplication
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}

@RestController
public class HealthController {
    @GetMapping("/health")
    public String health() { return "OK"; }
}
```

What happens when this starts: the JVM boots, `SpringApplication.run` creates the `ApplicationContext`, component scanning discovers `HealthController`, auto-configuration wires up an embedded Tomcat and Spring MVC's dispatcher servlet, and the app begins listening on port 8080 — all from that one `main` method.

Configuration: application.yml

```
server:
  port: 8081
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:postgresql://localhost:5432/orders
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate
  logging:
    level:
      com.example: DEBUG
```

```
@ConfigurationProperties(prefix = "app.mail")
public record MailProperties(String host, int port, boolean sslEnabled) {}
```

`@ConfigurationProperties` binds a whole configuration section to a strongly-typed record/class, instead of scattering `@Value("${...}")` injections everywhere.

Profiles & Actuator

```
# application-prod.yml overrides application.yml when profile=prod is active
spring:
  jpa:
    show-sql: false
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
# exposes /actuator/health, /actuator/metrics, /actuator/info out of the box
```

Actuator exposes production-ready endpoints for health checks and metrics with zero custom code — the foundation Part 29 builds monitoring on top of. **DevTools** adds automatic restart on code changes during local development.

INTERVIEW QUESTIONS – PART 17

1. What happens, step by step, when a Spring Boot application starts?
2. How does auto-configuration decide what to configure?
3. What three annotations does `@SpringBootApplication` combine?
4. How would you override an auto-configured bean with your own?
5. What's the difference between `application.yml` and `application-prod.yml`, and how do you activate the latter?
6. What does Actuator give you out of the box, and why does that matter in production?

Next: Part 18 — REST API Development with Spring Boot: controllers, DTOs, validation, and building real APIs end to end.

Part 18 — REST API Development with Spring Boot

Controllers · DTOs · Validation · ResponseEntity · Global exception handling · Pagination & filtering · Building a Product API

Layered architecture, in code

```
src/main/java/com/example/products/
├─ controller/ProductController.java
├─ service/ProductService.java
├─ repository/ProductRepository.java
├─ entity/Product.java
├─ dto/ProductRequest.java, ProductResponse.java
└─ exception/ProductNotFoundException, GlobalExceptionHandler.java
```

Entity, DTOs, and why they're separate

```
@Entity
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private BigDecimal price;
    private Integer stock;
    // getters/setters omitted
}

public record ProductRequest(
    @NotBlank String name,
    @Positive BigDecimal price,
    @Min(0) Integer stock
) {}

public record ProductResponse(Long id, String name, BigDecimal price, Integer stock) {}
```

The **entity** mirrors the database table; **DTOs** (Data Transfer Objects) shape exactly what the API accepts and returns. Keeping them separate means internal schema changes don't automatically break the public API contract, and lets validation rules live only on the input side.

The controller

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public Page<ProductResponse> list(
        @RequestParam(required = false) String search,
        @PageableDefault(size = 20, sort = "name") Pageable pageable) {
        return productService.search(search, pageable);
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getById(id));
    }

    @PostMapping
    public ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest
request) {
        ProductResponse created = productService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductResponse> update(@PathVariable Long id, @Valid
@RequestBody ProductRequest request) {
        return ResponseEntity.ok(productService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Line by line: `@RequestMapping("/api/products")` sets the base path for every method below. `@PathVariable` extracts `{id}` from the URL; `@RequestParam` reads a query string parameter; `@RequestBody` deserializes the JSON body into the DTO; `@Valid` triggers the Bean Validation

annotations on that DTO before the method body runs. `ResponseEntity` lets you control the exact status code and headers returned, not just the body.

Global exception handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ProductNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException
ex) {
        String message = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR",
message));
    }
}

public record ErrorResponse(String code, String message) {}
```

One centralized `@RestControllerAdvice` catches exceptions thrown anywhere in the controller layer and converts them into consistent JSON error responses – no repeated try/catch in every endpoint.

COMMON MISTAKE

Returning your JPA `@Entity` directly from a controller leaks database structure into the API, risks lazy-loading exceptions when Jackson tries to serialize an unfetched relationship, and makes the API contract fragile to schema changes. Always map to a DTO at the boundary.

PRACTICE NOW

Build a complete Student API and an Employee API following this exact structure (Controller → Service → Repository → Entity/DTO), each with pagination, search-by-name filtering, and full CRUD with proper status codes. Add global exception handling covering "not found" and validation errors for both.

INTERVIEW QUESTIONS – PART 18

1. Why separate entities from DTOs instead of exposing entities directly?
2. What does @Valid do, and where does the actual validation logic run?
3. @PathVariable vs. @RequestParam – when do you use each?
4. How does @RestControllerAdvice centralize error handling?
5. What status code should a successful POST that creates a resource return, and why 201 rather than 200?
6. How would you implement pagination and sorting cleanly in a Spring Boot endpoint?
7. What is API versioning, and name two common strategies (URL path vs. header-based).

Next: Part 19 – Spring Data JPA & Hibernate: entities, relationships, and solving the N+1 problem.

Part 19 – Spring Data JPA & Hibernate

Entity mapping · Relationships · Fetch strategies · JPQL & Specifications · The N+1 problem

JPA, Hibernate, and JpaRepository

JPA (Jakarta Persistence API) is a specification for object-relational mapping (ORM) – mapping Java objects to database rows. **Hibernate** is the most common implementation of that specification. **Spring Data JPA** sits on top of both, generating repository implementations from interfaces so you rarely write JPQL or SQL by hand.

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartment(String department);           // derived query – Spring
    // writes the SQL
    Optional<Employee> findByEmail(String email);
    @Query("SELECT e FROM Employee e WHERE e.salary > :min")
    List<Employee> findHighEarners(@Param("min") BigDecimal min);
}
```

Entity relationships

```
@Entity
public class Department {
    @Id @GeneratedValue private Long id;
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
}

@Entity
public class Employee {
    @Id @GeneratedValue private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Relationship	Example	Default fetch
@OneToOne	User ↔ UserProfile	Eager
@OneToMany	Department → Employees	Lazy
@ManyToOne	Employee → Department	Eager
@ManyToMany	Student ↔ Course	Lazy

Lazy loading defers fetching related data until it's actually accessed; **eager** fetches it immediately with the parent. `cascade = CascadeType.ALL` propagates persist/remove operations from parent to children; `orphanRemoval = true` deletes a child automatically when it's removed from its parent's collection.

PRO TIP

Override the default eager fetch on @ManyToOne/@OneToOne to `FetchType.LAZY` almost everywhere in real applications – eager defaults are a common source of accidentally loading far more data than a request needs.

The N+1 problem

```
List<Department> departments = departmentRepository.findAll(); // 1 query
for (Department d : departments) {
    d.getEmployees().size(); // N additional queries – one per department!
}
```

This is the classic **N+1 query problem**: one query to fetch parents, then one more per parent to lazily fetch its children – 101 queries for 100 departments. Fix it with a **fetch join**:

```
@Query("SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees")
List<Department> findAllWithEmployees(); // now just 1 query total
```

DEBUG THIS

Enable `spring.jpa.show-sql=true` locally and count the actual queries a single "list departments" endpoint fires. Seeing a query count that scales with the number of returned rows is the signature of N+1 – fix it with a fetch join, an `@EntityGraph`, or a DTO projection query.

Specifications & projections

```
public class EmployeeSpecs {
    public static Specification<Employee> hasDepartment(String dept) {
        return (root, query, cb) -> cb.equal(root.get("department"), dept);
    }
}

List<Employee> result =
employeeRepository.findAll(EmployeeSpecs.hasDepartment("Engineering"));

public interface EmployeeSummary { String getName(); BigDecimal getSalary(); } //
projection – fetches only these columns
List<EmployeeSummary> findByDepartment(String department);
```

Specifications compose dynamic, type-safe query criteria at runtime – ideal for search/filter endpoints with many optional parameters. **Projections** fetch only the columns you need instead of the whole entity, reducing data transfer.

Entity lifecycle & transactions

```
Transient → (persist) → Managed → (remove) → Removed
      |
      | (detach / commit)
      ▼
    Detached
```

```
@Transactional
public void giveRaise(Long employeeId, BigDecimal amount) {
    Employee e = employeeRepository.findById(employeeId).orElseThrow();
    e.setSalary(e.getSalary().add(amount));
    // no explicit save() needed – a managed entity's changes are
    // auto-flushed to the DB when the transaction commits ("dirty checking")
}
```

`@Transactional` wraps the method in a database transaction: it commits automatically on success and rolls back on an unchecked exception. A **managed** entity (fetched within an active persistence context) is tracked by Hibernate – modifying its fields is enough; Hibernate detects the change ("dirty checking") and writes it on commit without an explicit save call.

INTERVIEW QUESTIONS – PART 19

1. What is the N+1 problem, and how do you both detect and fix it?
2. Lazy vs. eager fetching – what's the tradeoff, and what's the danger of `LazyInitializationException`?
3. What does `@Transactional` actually do at runtime?
4. How does Hibernate's dirty checking let you skip calling `save()` explicitly?
5. When would you use a `Specification` instead of a derived query method?
6. `cascade = CascadeType.ALL` and `orphanRemoval` – what's the difference?
7. Why might returning a projection interface be better than the full entity for a search endpoint?

Next: Part 20 – Spring Security: authentication, JWT, and building a complete auth system.

Part 20 – Spring Security

Authentication vs. authorization · Password hashing · JWT · Filters · CORS/CSRF · A complete JWT auth system

Authentication vs. authorization

	Answers	Example
Authentication	"Who are you?"	Verifying a username/password or a JWT signature
Authorization	"What can you do?"	Only an ADMIN role may call DELETE /api/users/{id}

Password hashing

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();    // never store plaintext passwords
}

String hashed = passwordEncoder.encode("mypassword");    // one-way hash, includes a random salt
boolean matches = passwordEncoder.matches("mypassword", hashed);
```

BCrypt is deliberately slow and includes a per-hash salt, making brute-force and rainbow-table attacks impractical even if the database leaks.

How Spring Security authenticates a request

```
Request → SecurityFilterChain → JwtAuthFilter (extracts & validates token)
→ sets Authentication in SecurityContext → Controller runs, with
principal/roles available → response
```

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, JwtAuthFilter jwtAuthFilter)
    throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless API – no CSRF
            risk from cookies
            .sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

JWT authentication, end to end

1. POST /api/auth/login {email, password}
2. Server verifies credentials, issues a signed JWT (+ refresh token)
3. Client stores the token, sends it as: Authorization: Bearer <token>
4. Every subsequent request: JwtAuthFilter validates the signature/expiry, loads the user, sets SecurityContext – no server-side session needed
5. Access token expires quickly (e.g. 15 min); the refresh token (longer-lived, stored more carefully) is used to obtain a new one

```

public class JwtService {
    private final SecretKey key = Keys.hmacShaKeyFor(secretBytes);

    public String generateToken(String username, List<String> roles) {
        return Jwts.builder()
            .subject(username)
            .claim("roles", roles)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 900_000))    // 15 min
            .signWith(key)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser().verifyWith(key).build()
            .parseSignedClaims(token).getPayload().getSubject();
    }
}

public class JwtAuthFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
    FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String username = jwtService.extractUsername(token);
            if (username != null && jwtService.isValid(token)) {
                var auth = new UsernamePasswordAuthenticationToken(username, null,
                authorities);
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}

```

CORS & CSRF

	What it protects against	Relevant when
CORS	Browsers blocking cross-origin requests by default – you configure which origins may call your API	Any browser-based frontend on a different domain/port than the API
CSRF	A malicious site tricking a logged-in user's browser into submitting an unwanted request	Cookie/session-based auth – largely irrelevant for stateless JWT APIs, which is why it's disabled above

COMMON MISTAKE

Disabling CSRF is correct for a stateless, token-based API — but doing the same for a traditional session-cookie-based app removes real protection. The rule is: CSRF matters when the browser *automatically* attaches credentials (cookies); it doesn't apply when the client must explicitly attach a bearer token itself.

INTERVIEW QUESTIONS – PART 20

1. Authentication vs. authorization — give a concrete example of each.
2. Why is BCrypt preferred over a plain SHA-256 hash for passwords?
3. Walk through what happens to a request carrying a JWT, from arrival to the controller running.
4. Why use a short-lived access token plus a longer-lived refresh token instead of one long-lived token?
5. Why is CSRF protection typically disabled for stateless JWT APIs?
6. What's stored in the SecurityContext, and how long does it live?
7. How would you restrict an endpoint to users with a specific role?

MINI CHALLENGE

Build a complete auth flow: `/api/auth/register` (hash + store password), `/api/auth/login` (verify + issue access/refresh tokens), `/api/auth/refresh` (exchange a valid refresh token for a new access token), and a protected `/api/users/me` endpoint that reads the authenticated principal from the SecurityContext.

Part 20 Quick Revision

Concept	Key takeaway
Password storage	Always BCrypt-hashed, never plaintext or reversible encryption
JWT	Self-contained, signed token — no server-side session lookup needed
Stateless security	SessionCreationPolicy.STATELESS + a custom filter validating the token per request
CSRF	Relevant for cookie auth, not for bearer-token APIs

Next: Part 21 – Testing: JUnit 5, Mockito, MockMvc, and Testcontainers.

Part 21 – Testing

JUnit 5 · Mockito · MockMvc · @SpringBootTest · Testcontainers · Test strategy

The testing pyramid



E2E / manual tests

Integration tests (MockMvc, @SpringBootTest, Testcontainers)

Unit tests (JUnit + Mockito) – most numerous, fastest, cheapest

Unit testing with JUnit 5 & Mockito

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock private OrderRepository orderRepository;
    @InjectMocks private OrderService orderService;

    @Test
    void shouldThrowWhenOrderNotFound() {
        when(orderRepository.findById(1L)).thenReturn(Optional.empty());

        assertThrows(OrderNotFoundException.class, () -> orderService.getById(1L));
    }

    @Test
    void shouldCalculateTotalCorrectly() {
        Order order = new Order(List.of(new LineItem("A", 2, new BigDecimal("10.00"))));
        when(orderRepository.findById(1L)).thenReturn(Optional.of(order));

        BigDecimal total = orderService.getTotal(1L);

        assertEquals(new BigDecimal("20.00"), total);
    }
}
```

@Mock creates a fake implementation of a dependency; @InjectMocks injects those mocks into the class under test. when(...).thenReturn(...) stubs mock behavior, isolating the service logic from the real repository/database entirely – a true unit test.

Controller testing with MockMvc

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private ProductService productService;

    @Test
    void shouldReturn404WhenProductMissing() throws Exception {
        when(productService.getById(99L)).thenReturn(new ProductNotFoundException(99L));

        mockMvc.perform(get("/api/products/99"))
            .andExpect(status().isNotFound());
    }

    @Test
    void shouldCreateProduct() throws Exception {
        mockMvc.perform(post("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"name\":\"Widget\",\"price\":9.99,\"stock\":100}"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.name").value("Widget"));
    }
}
```

`@WebMvcTest` loads only the web layer (fast — no real database, no full application context), letting you verify routing, status codes, and JSON shape in isolation.

Integration testing with Testcontainers

```
@SpringBootTest
@Testcontainers
class ProductRepositoryIntegrationTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:16");

    @DynamicPropertySource
    static void configure(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired private ProductRepository productRepository;

    @Test
    void shouldPersistAndRetrieveProduct() {
        Product saved = productRepository.save(new Product("Widget", new
        BigDecimal("9.99"), 100));
        assertTrue(productRepository.findById(saved.getId()).isPresent());
    }
}
```

Testcontainers spins up a real, disposable PostgreSQL instance in Docker for the test run – far more trustworthy than an in-memory H2 substitute, since it exercises the actual database engine used in production.

PRO TIP

Write mostly fast unit tests (mocked dependencies) for business logic, a smaller set of MockMvc tests per controller for the web layer contract, and a handful of full Testcontainers integration tests for the critical end-to-end paths. Don't invert the pyramid – a suite of only slow integration tests punishes every commit.

INTERVIEW QUESTIONS – PART 21

1. Unit vs. integration test – what's actually different about what each exercises?
2. What does `@Mock` + `@InjectMocks` let you isolate in a service test?
3. Why is `@WebMvcTest` faster than a full `@SpringBootTest`?
4. Why prefer Testcontainers over an in-memory H2 database for integration tests?
5. What's the risk of a test suite shaped like an inverted pyramid?
6. How would you test a method annotated `@Transactional` that should roll back on failure?

Next: Part 22 – Microservices: service boundaries, communication, and resilience patterns.

Part 22 – Microservices

Monolith vs. microservices · Service boundaries · Inter-service communication · Resilience patterns

Monolith → modular monolith → microservices

	Deployment	Scaling	Complexity
Monolith	One deployable unit	Scale the whole app together	Low operational, high coupling risk as it grows
Modular monolith	One deployable, internally modularized	Same as monolith	Disciplined internal boundaries without distributed-systems cost
Microservices	Many independently deployable services	Scale each service independently	High – network calls, data consistency, observability all become harder

REAL-WORLD INSIGHT

Most teams should start with a well-organized modular monolith, not microservices. Splitting into services adds real operational cost (network failures, distributed transactions, deployment coordination) that only pays off once a team or scaling problem genuinely needs independent deployability.

Defining service boundaries

Good service boundaries follow business capabilities (Order Service, Inventory Service, Payment Service), not technical layers (a "database service" or "controller service"). Each service should own its own data – no other service reads its database directly.

Inter-service communication

```
public interface PaymentClient {
    @GetMapping("/api/payments/{orderId}")
    PaymentStatus getStatus(@PathVariable String orderId);
}

@FeignClient(name = "payment-service", url = "${payment.service.url}")
public interface PaymentFeignClient extends PaymentClient {}

// Usage in OrderService
PaymentStatus status = paymentFeignClient.getStatus(orderId);
```

OpenFeign lets you call another service as if invoking a local interface – the framework handles the actual HTTP call. Communication styles: synchronous REST (simple, but couples availability), or asynchronous messaging via Kafka (Part 24) for looser coupling and better resilience to a downstream outage.

Resilience4j patterns

```
@CircuitBreaker(name = "paymentService", sc-camel-fallback-method = "paymentFallback")
@Retry(name = "paymentService")
@TimeLimiter(name = "paymentService")
public CompletableFuture<PaymentStatus> checkPayment(String orderId) {
    return CompletableFuture.supplyAsync(() -> paymentFeignClient.getStatus(orderId));
}

private CompletableFuture<PaymentStatus> paymentFallback(String orderId, Throwable t) {
    return CompletableFuture.completedFuture(PaymentStatus.UNKNOWN); // graceful
    degradation
}
```

Pattern	Protects against
Circuit breaker	Hammering a failing downstream service – trips open after repeated failures, fails fast
Retry	Transient, self-resolving failures (a momentary blip)
Timeout	A slow downstream call hanging your thread indefinitely
Bulkhead	One failing dependency exhausting all your threads/connections and taking down unrelated calls too

Observability across services

Once a request crosses multiple services, a single centralized log file is no longer enough.

Distributed tracing (Part 29) attaches a shared trace-id to a request as it hops across services, so you can reconstruct the full call chain and see where time was spent or where it failed.

Centralized logging aggregates every service's logs into one searchable place.

INTERVIEW QUESTIONS – PART 22

1. When would you choose microservices over a modular monolith, honestly?
2. How should service boundaries be drawn?
3. What does a circuit breaker do, and what state machine does it follow (closed/open/half-open)?
4. Why can naive retries make an outage worse without a backoff strategy?
5. Synchronous REST vs. async messaging between services – tradeoffs?
6. Why is distributed tracing necessary once you have more than one service?

Next: Part 23 – Spring Cloud: Config Server, API Gateway, and service discovery.

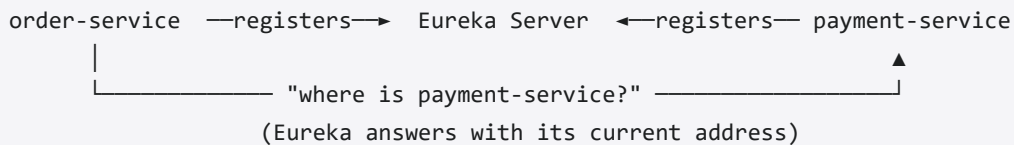
Part 23 – Spring Cloud

Config Server · API Gateway · Service discovery (Eureka) · Building a small microservices system

Why Spring Cloud

Spring Cloud is a collection of tools that solve cross-cutting distributed-systems problems on top of Spring Boot: where is a service running, where does its configuration live, and how do external clients reach many services through one entry point.

Service discovery with Eureka



```
@EnableEurekaServer
@SpringBootApplication
public class DiscoveryServerApplication { ... }

# order-service application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
spring:
  application:
    name: order-service
```

Instead of hardcoding `payment-service`'s host and port, other services ask Eureka for its current address — essential once services can be scaled up/down or moved, and their addresses change dynamically.

Config Server

```
@EnableConfigServer
@SpringBootApplication
public class ConfigServerApplication { ... }

# order-service bootstrap.yml
spring:
  config:
    import: optional:configserver:http://localhost:8888
```

A **Config Server** centralizes configuration for every service (often backed by a Git repository), so changing a shared property doesn't mean redeploying every service individually.

API Gateway

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
```

The **API Gateway** is the single entry point clients call; it routes each request to the correct backend service, and is the natural place to centralize cross-cutting concerns: authentication, rate limiting, logging, and request/response transformation.

```
Client → API Gateway → (routes by path) → order-service
                                         ↳ payment-service
                                         ↳ inventory-service
Config Server and Eureka support all of the above behind the scenes
```

INTERVIEW QUESTIONS – PART 23

1. What problem does service discovery (Eureka) solve that a hardcoded config file can't?
2. What's centralized in a Config Server, and why does that matter operationally?
3. What responsibilities does an API Gateway typically take on beyond simple routing?
4. How does OpenFeign combined with Eureka let you call another service by name instead of an IP?

Next: Part 24 – Kafka & Event-Driven Architecture.

Part 24 – Kafka & Event-Driven Architecture

Topics, partitions & offsets · Producers & consumers · Consumer groups · Error handling · Spring Kafka

Why event-driven architecture

In request/response communication, the caller waits and both services must be up simultaneously. In **event-driven architecture**, a service publishes a fact ("OrderPlaced") without knowing or caring who's listening – this decouples producers from consumers in both time and knowledge.

Kafka core concepts

Topic: "order-events"

┌────────── Partition 0 ─────────┐ offset: 0 1 2 3 4 →
┌────────── Partition 1 ─────────┐ offset: 0 1 2 3 →

Producer → writes to a partition (by key)

Consumer Group A: [consumer1 → P0, consumer2 → P1] (each partition read by one consumer per group)

Consumer Group B: [consumer3 → P0, P1] (an independent group reads everything again)

Term	Meaning
Topic	A named stream of events (like a table of facts)
Partition	An ordered, append-only log – a topic is split into partitions for parallelism
Offset	A message's position within its partition
Consumer group	A set of consumers sharing the work of reading a topic – each partition is read by exactly one consumer within a group

Producing and consuming with Spring Kafka

```
@Service
public class OrderEventProducer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderPlaced(Order order) {
        OrderEvent event = new OrderEvent(order.getId(), order.getTotal(), Instant.now());
        kafkaTemplate.send("order-events", order.getId().toString(), event);    // key =
order id
    }
}

@Component
public class OrderEventConsumer {
    @KafkaListener(topics = "order-events", sc-camel-group-id = "inventory-service")
    public void handle(OrderEvent event) {
        inventoryService.reserveStock(event.orderId());
    }
}
```

Sending with the order id as the **key** guarantees every event for the same order lands in the same partition, preserving per-order ordering even with multiple partitions.

Error handling & dead-letter topics

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);    // failed messages go
to order-events.DLT
    return new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3));    // retry 3x,
1s apart, then DLT
}
```

A **dead-letter topic** catches messages that repeatedly fail processing, so a single poison message doesn't block the whole partition forever – it's quarantined for manual inspection instead.

REAL-WORLD INSIGHT

Kafka guarantees at-least-once delivery by default – your consumer *will* occasionally see the same message twice (e.g. after a rebalance). Design consumers to be **idempotent** (safe to process the same event more than once) rather than assuming exactly-once delivery.

INTERVIEW QUESTIONS – PART 24

1. What's the difference between a Kafka topic and a partition?
2. How does a consumer group enable parallel processing while preserving per-partition order?
3. Why publish with a message key, and what does it control?
4. What is a dead-letter topic for?
5. Why must Kafka consumers be designed to be idempotent?
6. REST vs. Kafka for inter-service communication – when would you pick each?

Next: Part 25 – Redis & Caching.

Part 25 – Redis & Caching

Caching strategies · TTL & invalidation · Spring Cache · Redis integration · Distributed caching

Why cache

A cache stores the result of an expensive operation (a slow query, a computed report) in fast storage, so repeated requests skip the expensive path. **Redis** is an in-memory key-value store commonly used as a cache layer in front of a relational database.

Cache-aside pattern

```
read(id):
    value = cache.get(id)
    if value == null:
        value = database.get(id)    // cache miss – go to the source of truth
        cache.put(id, value, ttl)
    return value

write(id, value):
    database.save(id, value)
    cache.evict(id)                // invalidate – don't leave stale data cached
```

This is the **cache-aside** (lazy-loading) pattern: the application checks the cache first, falls back to the database on a miss, and populates the cache for next time. On write, the safest default is to evict the cache entry rather than try to update it in place.

Spring Cache abstraction

```
@EnableCaching
@Configuration
public class CacheConfig {}

@Service
public class ProductService {
    @Cacheable(value = "products", key = "#id")
    public ProductResponse getById(Long id) {
        return productRepository.findById(id).map(this::toResponse).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#id")
    public void update(Long id, ProductRequest request) { ... }

    @CachePut(value = "products", key = "#result.id()")
    public ProductResponse create(ProductRequest request) { ... }
}
```

`@Cacheable` checks the cache before running the method, and stores the result if it wasn't already there. `@CacheEvict` removes an entry (used on update/delete). `@CachePut` always runs the method and refreshes the cache with the result.

Wiring Redis

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 600000 # 10 minutes, in ms
```

TTL (time-to-live) automatically expires an entry after a set duration – the simplest, most robust invalidation strategy for data that's allowed to be briefly stale (product listings, dashboards) rather than data that must always be perfectly fresh (an account balance).

PRO TIP

In a multi-instance deployment, an in-memory (per-JVM) cache gives every instance a different, inconsistent view. Redis's shared, external cache means every application instance sees the same cached value – essential once you run more than one replica.

INTERVIEW QUESTIONS – PART 25

1. Explain the cache-aside pattern.
2. @Cacheable vs. @CachePut vs. @CacheEvict – when do you use each?
3. Why is TTL a simple, effective invalidation strategy – and when is it insufficient?
4. Why does a distributed cache like Redis matter once you scale to multiple instances?
5. What's the risk of caching data that changes frequently, and how would you mitigate it?

Next: Part 26 – Docker: containerizing a Spring Boot application and its dependencies.

Part 26 – Docker

Images & containers · Dockerfile for Spring Boot · Docker Compose · A full local environment

Images vs. containers

An **image** is a read-only, versioned snapshot of everything an application needs (code, runtime, libraries, OS layers). A **container** is a running instance of an image – isolated from the host and other containers, but sharing the host's kernel (lighter than a full virtual machine).

Dockerfile for a Spring Boot app

```
# Multi-stage build – keeps the final image small
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline          # cache dependencies in their own layer
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

PRO TIP

A **multi-stage build** uses one stage with the full Maven+JDK toolchain to compile, then copies only the resulting jar into a slim JRE-only final image – the shipped image doesn't carry Maven, source code, or build tools, cutting its size dramatically.

Essential commands

```
docker build -t order-service:1.0 .
docker run -p 8080:8080 order-service:1.0
docker ps                                # list running containers
docker logs -f <container-id>
docker exec -it <container-id> sh        # shell into a running container
docker volume create order-data          # persistent storage surviving container
restarts
```

Docker Compose — a full local environment

```
services:
  app:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/orders
      SPRING_DATA_REDIS_HOST: redis
    depends_on: [db, redis, kafka]

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: orders
      POSTGRES_PASSWORD: secret
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:
    image: redis:7-alpine

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0

volumes:
  pgdata:
```

```
docker compose up -d      # starts everything, in the background
docker compose down      # stops and removes containers (add -v to also remove volumes)
```

One command now brings up the application plus every dependency it needs — Postgres, Redis, and Kafka — networked together, with the app referring to them by service name (`db`, `redis`, `kafka`) rather than an IP address.

INTERVIEW QUESTIONS – PART 26

1. Image vs. container – what's the actual difference?
2. Why use a multi-stage Dockerfile for a Spring Boot app?
3. What does a Docker volume solve that the container's own filesystem doesn't?
4. How do containers in the same Compose file reach each other by name?
5. Why is a container more lightweight than a full VM?

Next: Part 27 – CI/CD & DevOps for Java Developers.

Part 27 – CI/CD & DevOps for Java Developers

CI/CD fundamentals · Build/test/deploy pipelines · Secrets · Release strategies

CI/CD fundamentals

Term	Meaning
Continuous Integration (CI)	Every push is automatically built and tested – problems surface within minutes, not weeks
Continuous Delivery (CD)	Every change that passes CI is automatically packaged and ready to deploy
Continuous Deployment	Every passing change is automatically deployed to production, with no manual gate

A complete pipeline with GitHub Actions

```
name: CI/CD
on:
  push:
    branches: [main]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: mvn -B verify

  build-and-push-image:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: $
          password: $
      - run: |
          docker build -t myorg/order-service:$ .
          docker push myorg/order-service:$

  deploy:
    needs: build-and-push-image
    runs-on: ubuntu-latest
    steps:
      - run: echo "Trigger deployment to staging/production here"
```

Each job runs only if the previous one (**needs:**) succeeded — tests must pass before an image is built, and an image must exist before deployment runs. **Secrets** (like registry credentials) are stored encrypted in the CI platform and injected as environment variables, never committed to the repository.

Release strategies

Strategy	How it works	Risk profile
Rolling deployment	Replace old instances with new ones gradually	Brief mixed-version window
Blue-green	Deploy the new version fully alongside the old, then switch traffic at once	Instant rollback (switch back), doubles infrastructure briefly
Canary	Send a small % of traffic to the new version first, expand gradually	Limits blast radius of a bad release

PRO TIP

Environment-specific configuration (database URLs, feature flags) should come from environment variables or a config server, never be hardcoded per-environment in the jar itself – one built artifact should be promotable unchanged from staging to production.

INTERVIEW QUESTIONS – PART 27

1. Continuous delivery vs. continuous deployment – what's the actual difference?
2. Why should tests run in CI before an image is even built?
3. Blue-green vs. canary deployment – when would you prefer each?
4. How should secrets be handled in a CI/CD pipeline?
5. Why should the same built artifact be promoted across environments instead of rebuilding per environment?

Next: Part 28 – Cloud Fundamentals: AWS basics and deploying a Spring Boot app.

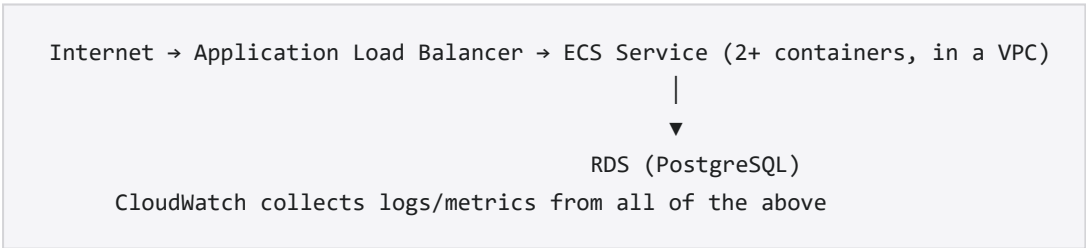
Part 28 – Cloud Fundamentals

AWS core services · Networking basics · Deploying a Spring Boot application

Core AWS services for a backend developer

Service	What it's for
EC2	Virtual machines you rent and manage – run your Spring Boot jar directly
ECS / ECR	Container orchestration (ECS) and a private Docker image registry (ECR)
RDS	Managed relational database (PostgreSQL/MySQL) – handles backups, patching
S3	Object storage for files (uploads, static assets, backups)
IAM	Identity and Access Management – who/what can access which AWS resources
VPC	An isolated private network for your resources
Load Balancer (ALB)	Distributes incoming traffic across multiple app instances
CloudWatch	Logs, metrics, and alarms

A typical deployment topology



Running at least two container instances behind a load balancer means one instance can fail or be redeployed without downtime – the load balancer simply stops routing to it.

IAM & least privilege

IAM roles/policies define exactly what each service is allowed to do – e.g. the application's role might allow reading from one specific S3 bucket and nothing else. **Least privilege**: grant only the permissions a resource genuinely needs, nothing broader.

Deploying, conceptually

1. Build the Spring Boot jar and Docker image (Part 26/27's pipeline)
2. Push the image to ECR
3. Update the ECS service to use the new image tag
4. ECS performs a rolling deployment, replacing old containers with new ones behind the load balancer
5. The app connects to RDS via a connection string injected from environment variables/secrets, never hardcoded

INTERVIEW QUESTIONS – PART 28

1. Why use RDS instead of running your own PostgreSQL on an EC2 instance?
2. What does a load balancer protect you against operationally?
3. What is the principle of least privilege, and how does IAM enforce it?
4. What's the difference between a VPC and the public internet, conceptually?
5. Why would you run 2+ container instances instead of 1, even for a low-traffic app?

Next: Part 29 – Observability & Production: logging, metrics, tracing, and Actuator in depth.

Part 29 – Observability & Production

Structured logging · Metrics · Health checks · Prometheus & Grafana · Distributed tracing · Alerts

The three pillars of observability

Pillar	Answers	Tooling
Logs	"What exactly happened?"	Structured JSON logs, aggregated centrally
Metrics	"How is the system behaving, numerically, over time?"	Micrometer → Prometheus → Grafana
Traces	"Where did this specific request's time go, across services?"	OpenTelemetry, distributed tracing

Structured logging

```
log.info("Order placed", kv("orderId", order.getId()), kv("total", order.getTotal()));  
// {"level":"INFO","msg":"Order placed","orderId":42,"total":99.90,"traceId":"a1b2c3"}
```

Structured (JSON) logs are machine-parseable, letting you search/filter/aggregate by field (e.g. "all ERROR logs for sc-camel-order-id=42") instead of grepping free text.

Actuator, metrics, and Prometheus

```
management:  
  endpoints:  
    web:  
      exposure:  
        include: health, metrics, prometheus  
  metrics:  
    tags:  
      application: order-service
```

```
@Timed(value = "order.creation.time")
public Order createOrder(OrderRequest request) { ... }

Counter sc-camel-orders-placed = Counter.builder("orders.placed").register(registry);
ordersPlaced.increment();
```

/actuator/health reports whether the app (and its dependencies – database, disk space) is healthy; /actuator/prometheus exposes metrics in a format Prometheus scrapes on a schedule. **Grafana** then visualizes those metrics as dashboards, and can fire alerts when a metric crosses a threshold.

Distributed tracing

```
traceId: a1b2c3 (shared across every service the request touches)
├─ API Gateway [span: 5ms]
│   └─ order-service [span: 40ms]
│       └─ payment-service [span: 30ms] ← slow – the bottleneck
│           └─ inventory-service [span: 5ms]
```

OpenTelemetry auto-instruments Spring Boot to propagate a trace-id across every HTTP/messaging call, letting a tool like Jaeger or Grafana Tempo reconstruct exactly where a slow request spent its time.

Alerts & production debugging

Good alerts fire on symptoms users feel (elevated error rate, p99 latency spike, health check failing) – not on every internal fluctuation. Once alerted, the debugging loop is: check the dashboard for the affected timeframe → correlate with a deploy or traffic spike → drill into traces for a slow/failing request → cross-reference structured logs for that trace-id → fix and verify the metric recovers.

INTERVIEW QUESTIONS – PART 29

1. Logs vs. metrics vs. traces – what distinct question does each answer?
2. What does Spring Boot Actuator give you without writing custom code?
3. Why prefer structured (JSON) logs over plain text in production?
4. What does a trace-id let you do across microservices that logs alone can't?
5. What makes a good production alert vs. a noisy one?

Next: Part 30 – System Design for Java Developers.

Part 30 – System Design for Java Developers

Requirements · Scalability & availability · CAP theorem · Sharding · Case study: URL Shortener

The system design method

1. **Clarify functional requirements** – what must the system do? (e.g. "shorten a URL," "redirect to the original")
2. **Clarify non-functional requirements** – how well? (scale: reads/writes per second; latency target; availability target)
3. **Estimate scale** – back-of-envelope: users, requests/sec, storage growth
4. **High-level design** – draw the major components and how data flows
5. **Deep dive** – the interviewer's chosen bottleneck: the database schema, the hashing scheme, the caching layer
6. **Identify tradeoffs** – nothing is free; name what you're trading away

Core vocabulary

Term	Meaning
Scalability	The system's ability to handle growing load by adding resources
Availability	The fraction of time the system is up and serving requests correctly
Reliability	The system does the correct thing consistently, not just "is up"
Horizontal scaling	Add more machines
Vertical scaling	Make one machine bigger
Sharding	Splitting one dataset across multiple databases by a key, so no single database holds everything
Replication	Copying data across multiple nodes for redundancy and read scaling

CAP theorem

In the presence of a network **P**artition, a distributed system must choose between **C**onsistency (every read sees the latest write) and **A**vailability (every request gets a response, possibly stale). You cannot have all three simultaneously during a partition – most real systems pick a practical point between "strong" and "eventual" consistency based on the use case (a bank balance leans consistent; a social media like-count leans available).

Case study: design a URL shortener

Functional: shorten a long URL, redirect a short code to the original. **Non-functional:** very read-heavy (redirects vastly outnumber creations), low redirect latency, high availability.

```
Client → Load Balancer → App Servers → Cache (Redis) → Database
                        |                               |
                        generate short code             urls(id, short_code, long_url,
created_at)

write path: POST /shorten {longUrl} → generate code (base62 of an id, or hash) → store →
return short URL
read path:  GET /{code} → check cache → on miss, look up DB → 301/302 redirect →
populate cache
```

Decision	Choice & reasoning
Code generation	Base62-encode an auto-incrementing id (simple, no collisions) rather than hashing + collision-checking
Storage	A key-value store or simple relational table – the access pattern is a single-key lookup, not complex queries
Caching	Cache hot short codes in Redis – the read:write ratio is extremely skewed toward reads
Scaling reads	Add read replicas / more cache capacity before considering sharding, since writes are comparatively rare

PRACTICE NOW – DESIGN PROMPTS

Apply the same six-step method to: an e-commerce checkout system, a food delivery matching service, a ride-sharing dispatch system, a social media news feed, a payment processing system, a notification service (push/email/SMS fanout), a job portal search, a banking ledger, and a video streaming platform. For each: state the functional/non-functional requirements first, then draw the high-level component diagram before going deep on any one part.

INTERVIEW QUESTIONS – PART 30

1. Explain the CAP theorem with a concrete example of choosing C over A.
2. Horizontal vs. vertical scaling – tradeoffs of each?
3. What's the difference between sharding and replication?
4. Why would a URL shortener favor availability over strict consistency?
5. How would you estimate the storage needs for 100M shortened URLs?

Next: Part 31 – Design Patterns: creational, structural, and behavioral, each with a Java and Spring Boot example.

Part 31 – Design Patterns

Creational · Structural · Behavioral – each with the problem, the Java solution, and a Spring Boot connection

Creational patterns

Pattern	Problem	Solution
Singleton	Need exactly one instance of a class (e.g. one config object)	Private constructor + static accessor – Spring beans are singletons by default, so you rarely hand-write this
Factory	Object creation logic is complex or must vary by input	A method that returns different concrete types behind one interface
Builder	A constructor with many optional parameters becomes unreadable	Chain setter-style calls, then build() the final immutable object

```
// Factory
public interface NotificationSender { void send(String to, String msg); }
public class NotificationFactory {
    public static NotificationSender create(String channel) {
        return switch (channel) {
            case "email" -> new EmailSender();
            case "sms" -> new SmsSender();
            default -> throw new IllegalArgumentException("Unknown channel: " + channel);
        };
    }
}

// Builder
Order order = Order.builder()
    .customerId(1L)
    .addItem("Widget", 2)
    .shippingAddress(address)
    .build();
```

REAL-WORLD INSIGHT

Spring's `ApplicationContext` is itself a giant Factory + Singleton registry: `@Bean` methods are factory methods, and by default the container hands back the same singleton instance every time a bean is requested.

Structural patterns

Pattern	Problem	Solution
Adapter	Two incompatible interfaces need to work together	A wrapper class translating one interface's calls into another's
Decorator	Add behavior to an object without altering its class or affecting other instances	Wrap the object in another implementing the same interface, adding behavior around delegated calls
Facade	A subsystem has many complex parts callers shouldn't need to know about	One simple class exposing a small, high-level API over the complexity
Proxy	Need to control or intercept access to an object (lazy loading, security, logging)	A stand-in object implementing the same interface, forwarding to the real one

```
// Decorator
public interface DataSource { String read(); }
public class EncryptedDataSource implements DataSource {
    private final DataSource wrapped;
    public EncryptedDataSource(DataSource wrapped) { this.wrapped = wrapped; }
    public String read() { return decrypt(wrapped.read()); } // adds behavior around the
original
}
```

REAL-WORLD INSIGHT

Every Spring `@Transactional` or `@Cacheable` bean is wrapped in a dynamic **Proxy** at startup – calling the method actually calls the proxy first, which opens a transaction/checks the cache, then delegates to your real method. This is also exactly why calling a `@Transactional` method from another method *in the same class* doesn't trigger the transaction – the call bypasses the proxy entirely.

Behavioral patterns

Pattern	Problem	Solution
Strategy	An algorithm needs to vary independently of the client using it	Extract each variant behind a common interface, inject the chosen one
Observer	Multiple parts of the system need to react when something happens	Subjects notify a list of registered listeners – the basis of Spring's ApplicationEvent
Template Method	Several algorithms share the same skeleton but differ in specific steps	An abstract class defines the skeleton, subclasses override the varying steps
Chain of Responsibility	A request should pass through a series of handlers until one handles it	Each handler either processes the request or passes it to the next – this is literally how Spring Security's filter chain works

```
// Strategy
public interface DiscountStrategy { BigDecimal apply(BigDecimal price); }
public class PercentageDiscount implements DiscountStrategy {
    public BigDecimal apply(BigDecimal price) { return
price.multiply(BigDecimal.valueOf(0.9)); }
}

@Service
public class PricingService {
    private final DiscountStrategy discountStrategy; // injected – swap strategies
without touching this class
    public PricingService(DiscountStrategy discountStrategy) { this.discountStrategy =
discountStrategy; }
}

// Observer – Spring's built-in event mechanism
@Component
public class OrderPlacedListener {
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        emailService.sendConfirmation(event.getOrder());
    }
}
applicationEventPublisher.publishEvent(new OrderPlacedEvent(order));
```

INTERVIEW QUESTIONS – PART 31

1. Why are Spring beans singletons by default, and how does that relate to the Singleton pattern?
2. Decorator vs. Proxy – what's structurally different about their intent?
3. Explain how `@Transactional`'s proxy-based implementation explains the "self-invocation" gotcha.
4. Give a real use of the Strategy pattern in a Spring Boot app you might build.
5. How does Spring Security's filter chain resemble Chain of Responsibility?
6. When would you reach for Builder instead of a telescoping constructor?

Next: Part 32 – DSA for Java Backend Jobs: the core structures and 300+ coding problems.

Part 33 — Java Coding Interview Preparation

High-frequency questions across Core Java, OOP, Collections, Concurrency, and Java 8+, with interview-ready answers

Core Java & OOP

Q: Why is Java considered platform-independent, and what actually crosses platforms?

A: Source compiles to bytecode (not native machine code); any machine with a matching JVM can run that same bytecode. The JVM is platform-specific, but the bytecode it consumes is not.

Q: Why does Java pass everything "by value," even objects?

A: A reference variable holds a value — the memory address of an object. Passing it to a method copies that address, not the object. The method can mutate the object through the copied reference, but reassigning the parameter itself never affects the caller's variable.

Q: Why can't static methods be overridden?

A: Static methods are resolved at compile time based on the reference type (static/early binding), not the runtime object type — this is "method hiding," not overriding, and dynamic dispatch never enters the picture.

Q: equals() vs. hashCode() — what's the contract?

A: Equal objects (per equals()) must produce the same hashCode(); unequal objects may or may not. Breaking this contract corrupts hash-based collections like HashMap/HashSet.

Collections & Generics

Q: Why does ArrayList resizing cost $O(n)$ occasionally but still average $O(1)$ amortized for add()?

A: Most adds just place an element in existing capacity — $O(1)$. When capacity is exhausted, ArrayList allocates a new, larger array (typically 1.5×) and copies everything — $O(n)$, but this happens rarely enough that the average cost per add, across many adds, is still $O(1)$.

Q: Why does modifying a List while iterating with a for-each loop throw ConcurrentModificationException?

A: The enhanced for-loop uses an Iterator internally, which tracks a modification count. Structurally changing the list outside the iterator's own remove() invalidates that count, and the next next() call detects the mismatch and fails fast — better than silent, undefined behavior.

Q: What is type erasure's practical downside?

A: You cannot create a generic array (`new T[10]`), use `instanceof` against a parameterized type, or overload methods that only differ by generic type parameter — the compiler has already erased that information by the time these checks would run.

Concurrency

Q: Why is a double-checked locking singleton subtle to get right?

A: Without the field being `volatile`, another thread can observe a partially-constructed object due to instruction reordering — the reference can appear non-null before the constructor has finished running.

```
private static volatile Singleton instance;
public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) instance = new Singleton();    // double-checked
        }
    }
    return instance;
}
```

Q: Why is `ConcurrentHashMap` faster than a `synchronized HashMap` under contention?

A: A `synchronized HashMap` locks the entire map for every operation. `ConcurrentHashMap` partitions its internal structure so different threads can operate on different segments/buckets concurrently without blocking each other.

Java 8+ & Streams

Q: Why can a stream only be consumed once?

A: A stream isn't a data structure — it's a one-time pipeline of operations over a source. Once a terminal operation runs, the pipeline is considered consumed; reusing it throws `IllegalStateException` by design, encouraging you to think in terms of transformations, not stored state.

Q: When is a parallel stream actually worth using?

A: Only for large datasets with CPU-bound, independent (no shared mutable state) per-element work — for small collections or I/O-bound work, the overhead of splitting/merging across threads usually outweighs any benefit.

RECRUITER PERSPECTIVE

Interviewers care less about reciting a definition and more about whether you can trace the "why" behind it – for every answer above, be ready to give the follow-up code example unprompted, the way each is shown here.

Next: Part 34 – Spring Boot Interview Preparation: architecture, debugging, and production-scenario questions.

Part 34 – Spring Boot Interview Preparation

Architecture, debugging, and production-scenario questions with strong answers

Architecture & internals

Q: How does Spring Boot auto-configuration actually decide what to configure?

A: `@EnableAutoConfiguration` loads a list of candidate configuration classes, each guarded by conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`). At startup, Spring evaluates each condition against your classpath and existing bean definitions, applying only the configurations that match – and backing off automatically wherever you've defined your own bean of the same type.

Q: What happens, in order, when a Spring Boot application starts?

A: The JVM starts → `main()` calls `SpringApplication.run()` → the `ApplicationContext` is created → component scanning discovers beans → auto-configuration applies matching configurations → beans are instantiated and dependencies injected → `@PostConstruct` hooks run → for a web app, the embedded server starts listening.

Q: How does `@Transactional` work internally, and why doesn't self-invocation trigger it?

A: Spring wraps the bean in a dynamic proxy at startup. An external call goes through the proxy, which opens a transaction, delegates to the real method, then commits/rolls back based on the outcome. A method calling another `@Transactional` method on `this` bypasses the proxy entirely (it's a direct call, not a call through the bean reference) – so the annotation is silently ignored.

Security & persistence

Q: How does Spring Security authenticate a typical JWT-based request?

A: A custom `OncePerRequestFilter` runs before the request reaches your controller; it extracts the `Authorization: Bearer` header, validates the token's signature and expiry, loads the user's authorities, and populates the `SecurityContext`. Later, `@PreAuthorize`/route-based rules check that context to allow or reject the request.

Q: How does Hibernate manage entities across their lifecycle?

A: An entity is `Transient` until `persist()`-ed, becoming `Managed` and tracked by the persistence context – changes to a managed entity are auto-detected ("dirty checking") and flushed on

commit. It becomes Detached once the persistence context closes (e.g. after the transaction ends), at which point changes are no longer tracked; `remove()` transitions it to Removed.

Q: How do you solve the N+1 problem?

A: Detect it via SQL logging (query count scaling with row count), then fix with a JPQL fetch join, an `@EntityGraph`, or a batch-size hint — replacing "1 query + N lazy queries" with a single query that fetches everything needed upfront.

Scenario-based

Q: A production endpoint is suddenly slow only under load. How do you investigate?

A: Check metrics/dashboards for the time window (CPU, memory, DB connection pool saturation, GC pauses). Check for an N+1 query newly introduced. Check thread dumps for blocked threads (a lock contention or a slow downstream dependency). Check whether a circuit breaker/timeout is missing on a call to a degraded dependency, causing thread starvation upstream.

Q: How would you design a scalable REST API for a system expecting 10x traffic growth?

A: Stateless services behind a load balancer (horizontal scaling), a cache layer for hot reads, pagination on all list endpoints, async processing (Kafka) for non-critical-path work, database read replicas, and connection pooling sized to the expected concurrency — validated with load testing before the growth actually arrives.

DEBUGGING-BASED

Q: A bean fails to start with `BeanCreationException: Circular reference`. What's happening and how do you fix it?

A: Two beans depend on each other via constructor injection, so neither can be fully constructed first. Fix by breaking the cycle — typically by extracting the shared responsibility into a third bean, or (last resort) using setter injection for one side to defer the dependency until after construction.

Next: Part 35 — Real-World Projects: ten complete, portfolio-ready builds.

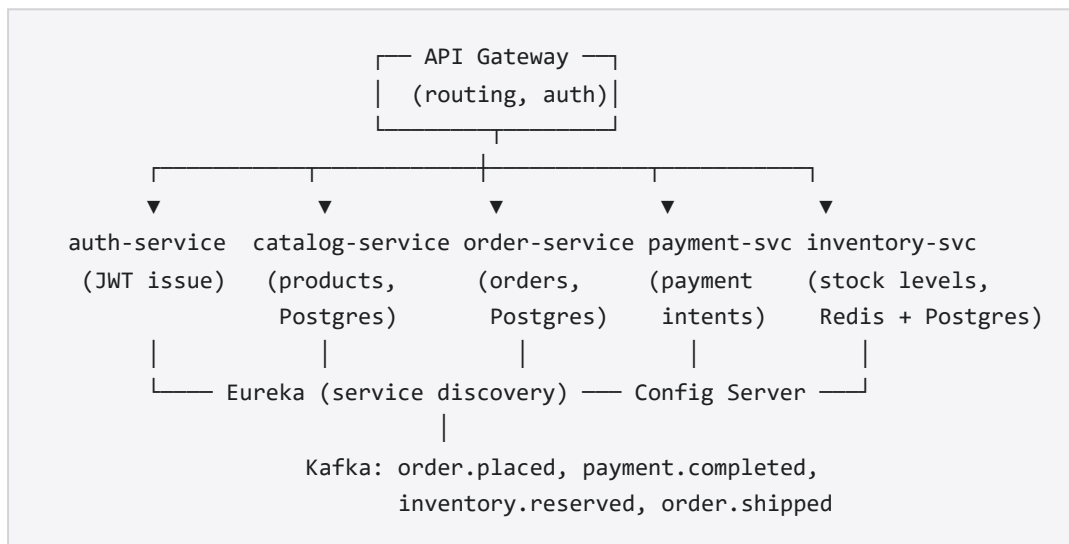
Part 36 – Capstone Project

One production-style enterprise platform, combining every part of this course

The brief: a multi-vendor marketplace platform

A marketplace where vendors list products, customers place orders, and payments and inventory are coordinated across independently deployable services – deliberately chosen because it needs nearly every skill in this course to build honestly.

Service architecture



How the pieces connect

Concern	Component	Course reference
Authentication	auth-service issues JWTs; every other service validates them via a shared filter	Part 20
Data per service	Each service owns its own schema – order-service never queries catalog-service's database directly	Part 19, Part 22
Order → payment → inventory flow	order-service publishes <code>order.placed</code> ; payment-service and inventory-service consume it independently and publish their own completion events	Part 24
Resilience	Circuit breakers around any remaining synchronous calls (e.g. catalog lookup during order creation)	Part 22
Caching	Product catalog reads cached in Redis; invalidated on vendor updates	Part 25
Deployment	Each service containerized, orchestrated via Docker Compose locally / ECS in the cloud	Part 26, 28
CI/CD	Each service has its own pipeline: test → build image → push → deploy	Part 27
Observability	Structured logs + Actuator metrics per service, correlated by a shared trace-id	Part 29
Testing	Unit tests per service, contract/integration tests with Testcontainers for the database, and an end-to-end smoke test hitting the gateway	Part 21

Suggested build order

1. auth-service – get JWT issuance and validation working standalone first
2. catalog-service – CRUD + search + Redis caching
3. order-service – depends on catalog (sync call) and publishes events
4. inventory-service and payment-service – consume order events, publish their own
5. Wire in Eureka + Config Server once 3+ services exist and coordination pain is real

6. Add the API Gateway last, once you know what routes actually need to be exposed
7. Layer in observability and CI/CD throughout, not as an afterthought

RECRUITER PERSPECTIVE

This is the single strongest portfolio piece from the whole course — it demonstrates you can reason about service boundaries, data ownership, eventual consistency, and operational concerns, not just CRUD. Be ready to explain honestly what you'd change with more time (that self-awareness reads better than pretending it's flawless).

Next: Part 37 — Debugging & Troubleshooting: a handbook of common production errors and how to fix them.

Part 37 – Debugging & Troubleshooting Handbook

Symptom → cause → diagnosis → fix → prevention, for the errors you'll actually hit

Error	Common cause	Fix
NullPointerException	Calling a method on a reference that was never assigned, or a lazy-loaded relationship accessed outside its session	Use Optional at API boundaries; add null checks at the edges of the system, not everywhere internally
BeanCreationException	A required bean is missing, misconfigured, or there's a circular dependency	Read the "Caused by" chain – it names the actual missing/circular bean; check @Profile/@ConditionalOn* guards
Port already in use	A previous instance of the app (or another process) still holds the port	<code>lsof -i :8080</code> (Mac/Linux) then kill the process, or change <code>server.port</code>
Database connection failures	Wrong URL/credentials, DB not started yet, connection pool exhausted	Verify the connection string, check the DB is reachable with a plain client first, check HikariCP pool size vs. concurrent load
LazyInitializationException	Accessing a lazy relationship after the persistence context/session has closed	Fetch what you need within the transactional method, use a fetch join/@EntityGraph, or map to a DTO before returning

Transaction rolled back unexpectedly	An unchecked exception was thrown inside a <code>@Transactional</code> method (this is by design) or the transaction spans a self-invoked call	Confirm the exception is expected to roll back; check for the self-invocation proxy gotcha (Part 34)
JWT errors (401 with valid-looking token)	Clock skew, wrong signing key, or expired token	Log the specific JWT exception type; verify server clocks are synced (NTP); confirm the signing secret matches across services
CORS errors	Frontend origin not in the allowed-origins list, or preflight (OPTIONS) not handled	Configure a <code>CorsConfigurationSource</code> explicitly rather than a wildcard in production
401 vs. 403	401: not authenticated at all. 403: authenticated but lacking permission	Return the correct one – conflating them confuses API consumers debugging their own auth flow
409 Conflict	A uniqueness constraint violated, or optimistic-locking version mismatch	Catch the specific <code>DataIntegrityViolationException/OptimisticLockException</code> and translate to a clear 409 response
500 with no clear cause in logs	An exception swallowed somewhere, or logging level too high to show it	Ensure <code>GlobalExceptionHandler</code> logs the full stack trace before responding; never silently catch Exception

Slow SQL / N+1 queries	Missing index, or lazy-loaded collections triggering per-row queries	EXPLAIN ANALYZE the query; enable SQL logging to count actual queries per request; add a fetch join
OutOfMemoryError in production	A memory leak (unbounded cache, unclosed resources) or heap sized too small for load	Heap dump + profiler diff between two points in time; check for ever-growing static collections
Docker container exits immediately	The main process crashed on startup — often a missing env var or bad config	<code>docker logs <id></code> immediately after the exit; run the image interactively with <code>docker run -it --entrypoint sh</code> to inspect
Kafka consumer stuck / not receiving messages	Wrong topic name, consumer group already fully caught up, or a poison message blocking the partition	Check consumer lag with a Kafka admin tool; verify group-id and topic spelling; check for a dead-letter scenario

DEBUG THIS — A GENERAL METHOD

1) Reproduce reliably — a bug you can't reproduce, you can't verify fixed. 2) Read the full stack trace bottom-to-top for the failure origin, top-to-bottom for the call chain. 3) Form one specific hypothesis at a time — don't shotgun random changes. 4) Add the minimum logging/breakpoint needed to confirm or reject it. 5) Fix the root cause, not the symptom — then add a test that would have caught it.

Next: Part 38 — Performance Optimization: JVM, database, and API tuning with before/after examples.

Part 38 – Performance Optimization

JVM tuning · Database & Hibernate optimization · API & caching · Before/after examples

The optimization discipline

Measure first, optimize second. Guessing at bottlenecks wastes effort on code that was never actually slow – profile or benchmark to find the real cost center before changing anything.

Database & Hibernate – before/after

```
// BEFORE – N+1 queries (1 + 100 for 100 departments)
List<Department> depts = departmentRepository.findAll();
depts.forEach(d -> d.getEmployees().size());

// AFTER – 1 query total, via fetch join
@Query("SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees")
List<Department> findAllWithEmployees();
```

```
// BEFORE – SELECT * pulls unused columns across the network
List<Employee> findByDepartment(String dept);

// AFTER – projection fetches only what the view needs
interface EmployeeNameOnly { String getName(); }
List<EmployeeNameOnly> findByDepartment(String dept);
```

```
-- BEFORE: Seq Scan on orders (cost=... rows=500000)
SELECT * FROM orders WHERE customer_id = 42;

-- AFTER: adding the missing index
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
-- now: Index Scan using idx_orders_customer_id (cost=... rows=12)
```

Collections & string handling – before/after

```
// BEFORE – O(n²) string concatenation
String result = "";
for (String s : items) result += s + ",";

// AFTER – O(n)
StringBuilder sb = new StringBuilder();
for (String s : items) sb.append(s).append(",");
```

```
// BEFORE – ArrayList.contains() is O(n) per call, O(n²) overall in a loop
List<String> sc-camel-block-list = loadBlockList();
for (String user : users) if (blockList.contains(user)) flag(user);

// AFTER – HashSet.contains() is O(1)
Set<String> sc-camel-block-list = new HashSet<>(loadBlockList());
```

API & caching optimizations

Technique	Effect
Pagination on every list endpoint	Bounds response size and query cost regardless of table growth
Cache-aside with Redis on hot, rarely-changing reads	Removes repeated database round trips for the same data
Connection pool sizing (HikariCP)	Too small starves throughput under load; too large exhausts the database's own connection limit – size to $(\text{core_count} \times 2) + \text{disk spindles}$ as a starting heuristic, then measure
Async processing for non-critical-path work (emails, notifications)	Keeps the request/response cycle fast by moving slow, non-essential work off it (via Kafka or @Async)

JVM tuning basics

```
java -Xms1g -Xmx1g -XX:+UseG1GC -jar app.jar
# Set Xms == Xmx in production to avoid runtime heap resizing pauses.
# Monitor GC pause frequency/duration via Actuator + a GC log before changing flags further.
```

INTERVIEW QUESTIONS – PART 38

1. Walk through your process for diagnosing a slow API endpoint.
2. Why measure before optimizing?
3. Give an example where adding a cache made things worse.
4. How would you size a database connection pool?
5. Name three concrete fixes for an N+1 query problem.

Next: Part 39 – Clean Code & Professional Development.

Part 39 – Clean Code & Professional Development

SOLID, DRY, KISS, YAGNI · Naming & structure · Refactoring · Code review

The core principles

Principle	Meaning
DRY	Don't Repeat Yourself – duplicated logic means duplicated bugs and duplicated future edits
KISS	Keep It Simple – the simplest design that solves the actual problem beats a clever one
YAGNI	You Aren't Gonna Need It – don't build flexibility for a requirement that doesn't exist yet
SOLID	See Part 12 – the five principles governing well-structured object-oriented design

Bad code → improved code

```
// BEFORE – unclear names, mixed responsibilities, magic numbers
public class Mgr {
    public double calc(double a, int t) {
        if (t == 1) return a * 0.9;
        if (t == 2) return a * 0.85;
        return a;
    }
}

// AFTER – intention-revealing names, extensible via polymorphism, no magic numbers
public interface DiscountPolicy { BigDecimal apply(BigDecimal price); }

public class LoyaltyDiscount implements DiscountPolicy {
    private static final BigDecimal LOYALTY_RATE = new BigDecimal("0.90");
    public BigDecimal apply(BigDecimal price) { return price.multiply(LOYALTY_RATE); }
}

public class BulkDiscount implements DiscountPolicy {
    private static final BigDecimal BULK_RATE = new BigDecimal("0.85");
    public BigDecimal apply(BigDecimal price) { return price.multiply(BULK_RATE); }
}
```

The "after" version names each concept explicitly, and adding a new discount type means adding a class – not editing a growing if-chain (Open/Closed Principle from Part 12).

Naming & method size

Weak	Better	Why
<code>process(d)</code>	<code>calculateMonthlyInterest(deposit)</code>	The name states what and on what, no need to read the body
<code>List<String> list1</code>	<code>List<String> activeUserEmails</code>	Type-based names carry no domain meaning
A 200-line method doing five things	Five well-named private methods, one calling the others	Each piece is independently readable, testable, and reusable

Refactoring & code review

Refactor in small, safe steps under an existing test suite — never rewrite untested code from memory. When reviewing a pull request, focus feedback on correctness, readability for the next reader, and adherence to the codebase's existing conventions — not personal style preference. Leave specific, actionable comments ("extract this into a named method" beats "this is messy").

PRO TIP

Good API/documentation habits compound: a clear README explaining setup and endpoints, meaningful commit messages (Part 15), and Javadoc on any genuinely non-obvious public method save far more time than they cost, especially once more than one person touches the code.

INTERVIEW QUESTIONS — PART 39

1. What's the practical difference between DRY and premature abstraction?
2. How would you refactor a long if/else chain based on a type field?
3. What makes a code review comment actionable vs. just critical?
4. Give an example of YAGNI — a "flexible" design that turned out to be unnecessary.

Next: Part 40 — Job Preparation: roadmap, resume, portfolio, and applications.

Part 40 – Job Preparation

Skills checklist · Resume & ATS · GitHub & LinkedIn · Applications & networking

The Java backend developer roadmap, at a glance

Stage	Focus
Foundation	Core Java, OOP, collections, exceptions (Parts 1–9)
Data	SQL, JDBC (Parts 10–12)
Web	HTTP fundamentals, Spring, Spring Boot, REST, JPA, Security, Testing (Parts 13–21)
Distributed systems	Microservices, Kafka, Redis, Docker, CI/CD, Cloud (Parts 22–29)
Interview-ready	System design, patterns, DSA, mock interviews (Parts 30–34, 41–42)
Portfolio	Real projects + capstone (Parts 35–36)

Resume & ATS optimization

- Lead each bullet with an action verb and a measurable outcome: "Reduced API p95 latency by 40% by fixing an N+1 query and adding Redis caching" beats "Worked on backend performance."
- Mirror the exact technology names from the job posting (Spring Boot, not "Spring framework"; PostgreSQL, not "SQL database") – ATS keyword matching is literal.
- List 2–3 real projects with a one-line description, tech stack, and a GitHub link each – quality over quantity.
- Keep it to one page for under 5 years of experience; a clean, single-column layout parses more reliably through ATS software than multi-column designs.

GitHub portfolio

Pin your 3–4 strongest projects (the capstone and 2–3 from Part 35). Each repo needs: a clear README (what it does, tech stack, how to run it locally, a screenshot or API example), meaningful commit history (not one giant "final commit"), and passing CI badge if you set up GitHub Actions.

LinkedIn & networking

- Headline states the target role plainly: "Backend Developer — Java, Spring Boot, PostgreSQL" reads better to a recruiter search than a vague tagline.
- Post about what you build — a short writeup of the capstone project's architecture decisions is genuinely interesting content and a natural conversation starter.
- A warm referral consistently outperforms a cold application — reach out to alumni, past colleagues, or people at target companies with a specific, short ask, not a generic "any tips?"

Presenting a project in an interview

Structure: the problem it solves (30 sec) → architecture at a high level, drawn if possible (1 min) → one technical decision worth defending, and why (1 min) → what you'd improve with more time (30 sec). This shows judgment, not just output.

PRO TIP

Apply broadly but track quality: a spreadsheet of company, role, date applied, referral status, and next action beats a memory-based approach once you're applying to more than a handful of roles.

Next: Part 41 — Mock Interviews across Java, Spring Boot, SQL, Microservices, and System Design.

Part 41 – Mock Interviews

Representative interviewer questions and ideal candidate answers, across five tracks

Track 1 – Java

Interviewer: "Your service is throwing intermittent `ConcurrentModificationExceptions`. Walk me through how you'd find and fix it."

Candidate: "That points to a collection being structurally modified while it's being iterated – likely a shared `List` or `Map` being read in one thread while another mutates it, or a for-each loop calling `remove()` directly on the collection instead of the iterator. I'd grep for iteration over that collection, check if it's ever accessed from more than one thread, and either switch to a concurrent collection like `CopyOnWriteArrayList`/`ConcurrentHashMap`, or use the iterator's own `remove()` method and add synchronization if multiple threads are genuinely involved."

Interviewer: "Why would you choose composition over inheritance here?"

Candidate: "Inheritance locks the subclass to one parent's implementation permanently and can violate Liskov Substitution if the subclass doesn't behave consistently with the parent's contract. Composition – giving the class a reference to a collaborator – keeps the relationship swappable and testable, which matters especially once the class needs to be unit tested in isolation."

Track 2 – Spring Boot

Interviewer: "A new developer added a bean with `@Autowired` on a field and it broke a unit test. Why, and how would you fix it?"

Candidate: "Field injection means the class has no public constructor accepting its dependency, so a plain unit test (not using Spring's context) can't supply a mock – the field stays null. I'd refactor to constructor injection: now the dependency is a required constructor parameter, trivially mockable with `new ServiceUnderTest(mockDependency)` with no Spring context needed at all."

Interviewer: "How would you add rate limiting to a public API?"

Candidate: "At the gateway level ideally, using a token-bucket algorithm keyed by client id or IP – Resilience4j has a `RateLimiter` module, or Redis-backed counters work well across multiple instances since the limit state needs to be shared, not per-instance."

Track 3 – Backend/General

Interviewer: "How do you decide when a synchronous REST call is fine vs. when you need messaging?"

Candidate: "If the caller genuinely needs the result immediately to proceed, and the two services' availability is already coupled anyway, REST is simpler and I'd default to it. If the work can happen after the fact – a confirmation email, updating a search index, notifying another bounded context – messaging decouples the services' uptime and lets the receiver process at its own pace."

Track 4 – SQL

Interviewer: "Write a query to find customers who placed orders in every month of Q1."

Candidate walks through: group orders by customer and month, count distinct months per customer, filter for count = 3 – `SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31' GROUP BY customer_id HAVING COUNT(DISTINCT DATE_TRUNC('month', order_date)) = 3`, narrating the HAVING clause's role explicitly.

Track 5 – System Design

Interviewer: "Design a notification service that can send push, email, and SMS."

Candidate: clarifies scale and delivery guarantees first, then proposes a NotificationRequest queue (Kafka), a router that picks a channel-specific worker per message type, retry with exponential backoff and a dead-letter queue for permanent failures, and idempotency keys so a retried send doesn't double-notify a user – explicitly naming the tradeoff of at-least-once delivery.

PRACTICE NOW

Record yourself answering each question above out loud, in under 90 seconds, without reading – then compare to the answer here. The gap between what you said and what's written is exactly what to drill next.

Next: Part 42 – Company-Style Interview Preparation: product, service, startup, and FAANG-style patterns.

Part 42 – Company-Style Interview Preparation

Product vs. service vs. startup vs. enterprise vs. FAANG-style loops

How interview loops differ by company type

Company type	Typical rounds	What they weight most
Product companies	DSA coding, system design, Java/Spring depth, behavioral	Problem-solving depth and system design judgment
Service/consulting companies	Core Java/SQL fundamentals, a practical coding test, communication round	Breadth of practical knowledge, client-facing communication
Startups	A take-home or pairing exercise close to their actual stack, culture fit	Can you ship, and do you fit the team's pace and ownership style
Enterprise companies	Structured rounds: Java fundamentals, Spring/JEE ecosystem, SQL, sometimes a formal design document review	Process discipline, deep framework knowledge, stability
FAANG-style	2–4 DSA coding rounds, 1 system design (for 3+ YOE), 1–2 behavioral	Algorithmic rigor and clear communication of tradeoffs under time pressure

Coding round expectations

Regardless of company type: think aloud, clarify constraints and edge cases before coding, start with a working brute-force solution if the optimal approach isn't obvious immediately, then optimize and state the resulting time/space complexity unprompted.

System design round expectations

Follow the Part 30 method explicitly: clarify requirements before drawing anything, state assumptions out loud, sketch the high-level design, then let the interviewer steer the deep dive – don't over-engineer a component they haven't asked about.

Behavioral/HR round

Use the STAR structure (Situation, Task, Action, Result) for every behavioral answer. Prepare concrete stories in advance for: a conflict with a teammate, a mistake you made and how you recovered, a time you pushed back on a decision, and a project you're proud of — reuse and adapt these across different phrased questions rather than improvising each time.

RECRUITER PERSPECTIVE

Ask the recruiter directly what the loop structure looks like before your interviews — most will tell you plainly, and tailoring your prep to the actual format (heavy DSA vs. heavy system design vs. take-home) is a far better use of limited prep time than guessing.

INTERVIEW QUESTIONS — PART 42

1. How would you adjust your prep for a service-company interview vs. a FAANG-style one?
2. Describe a time you disagreed with a technical decision (practice this using STAR).
3. What do you do when you're stuck on a coding problem mid-interview?

Next: Part 43 — The 90-Day Job-Ready Roadmap, day by day.

Part 43 – The 90-Day Job-Ready Roadmap

A week-by-week plan mapping directly onto Parts 1–42 of this course

Days 1–30 – Java + DSA + SQL

Week	Theory (this course)	Daily practice	Outcome
1	Parts 1–3 (Foundations, Java syntax, OOP basics)	10 beginner exercises/day (Part 1's list)	Comfortable writing basic Java programs
2	Parts 4–6 (Collections, Exceptions, Java 8+)	5 collections + 5 stream exercises/day	Fluent with Streams and core collections
3	Part 7 (Concurrency), Part 10–11 (Databases, SQL)	5 SQL queries/day against a practice schema	Can write joins, subqueries, and window functions confidently
4	Part 32 (DSA) – arrays, strings, two pointers, sliding window	3 Easy DSA problems/day	15–20 Easy problems solved independently

Days 31–60 – Spring + Spring Boot + JPA + REST + Security

Week	Theory	Daily practice	Outcome
5	Parts 16–17 (Spring, Spring Boot)	Build a "Hello API" from scratch each day, varying structure	Comfortable with DI, beans, and app startup
6	Part 18 (REST APIs)	Build the Student API and Employee API end to end	Two complete CRUD APIs with validation and error handling
7	Part 19 (JPA/Hibernate)	Add relationships + fix an intentionally introduced N+1 each day	Confident diagnosing and fixing N+1 issues
8	Parts 20–21 (Security, Testing)	Add JWT auth to a prior project; write 5 tests/day	One fully secured, tested API project

Days 61–75 — Microservices + Kafka + Redis + Docker

Week	Theory	Daily practice	Outcome
9	Parts 22–23 (Microservices, Spring Cloud)	Split one prior monolith project into two services	Two services communicating via OpenFeign + Eureka
10	Part 24 (Kafka)	Add an event published on order creation, consumed by a second service	Working event-driven flow between two services
11 (partial)	Parts 25–26 (Redis, Docker)	Cache one endpoint; containerize the whole stack with Compose	docker compose up brings up the full working system

Days 76–90 — Projects + System Design + Interviews + Resume + Applications

Days	Focus	Outcome
76–82	Finish and polish the Capstone (Part 36); write its README and Dockerize it	One flagship portfolio project, fully documented
83–85	System design practice (Part 30) — 1 design prompt/day, drawn and explained aloud	Comfortable structuring a 30–45 min design conversation
86–87	Mock interviews (Part 41) across all five tracks	Rehearsed, confident answers to the highest-frequency questions
88	Resume, GitHub, LinkedIn (Part 40)	Application-ready profile
89–90	Begin applying — 10+ applications, prioritizing referrals	Applications in flight, interview loop underway

PRO TIP

Don't wait until day 90 to start applying — begin sending applications around day 75 once the capstone is functionally complete, since interview loops themselves take 2–4 weeks to progress and you want that clock running in parallel with your final prep.

Next: Part 44 — The 100-Day Extended Roadmap, for learners who want a less compressed pace.

Part 44 — The 100-Day Extended Roadmap

The same curriculum at a gentler pace, with dedicated revision weeks

This plan covers identical ground to Part 43 but adds 10 extra days as two dedicated revision/catch-up weeks — useful if you're learning part-time alongside a job or studies, or want more repetition on DSA and system design before interviewing.

Days	Focus	Difference from the 90-day plan
1–35	Java + DSA + SQL (Parts 1–11, 32 basics)	+5 days: an extra revision week reworking every exercise you got wrong the first time
36–68	Spring + Spring Boot + JPA + REST + Security + Testing (Parts 16–21)	+3 days: extra time to rebuild the Student/Employee APIs from memory, without referring back to the notes
69–85	Microservices + Kafka + Redis + Docker (Parts 22–26)	+2 days: extra time debugging Docker Compose networking issues, which commonly trip up newcomers
86–100	Projects + System Design + Interviews + Resume + Applications (Parts 30, 35–36, 40–42)	Same structure as days 76–90 above, just with 5 extra prep days before applications begin

PRO TIP

Use each revision week for active recall, not re-reading: close the notes and rebuild a small project or re-solve a batch of DSA problems from scratch. Re-reading feels productive but builds far weaker retention than forcing yourself to reproduce the material unaided.

CHOOSING BETWEEN THE TWO PLANS

Pick the 90-day plan if you can dedicate 4+ focused hours daily and have some prior programming exposure. Pick the 100-day plan if you're learning part-time (1–2 hrs/day), are a complete beginner to programming, or know from past experience that you retain material better with built-in review cycles.

Next: Part 45 — Cheat Sheets & Rapid Revision: the final quick-reference and top-100 question sets, plus the Job-Ready Master Checklist.

Part 45 – Cheat Sheets, Rapid Revision & the Master Checklist

Quick-reference tables for every major topic, a glossary, and the final job-ready checklist

Cheat sheet – Java syntax & OOP

Need	Syntax
Class	<code>public class Foo { }</code>
Interface	<code>public interface Foo { void bar(); }</code>
Inheritance	<code>class Dog extends Animal { }</code>
Interface impl.	<code>class Foo implements Bar { }</code>
Record	<code>public record Point(int x, int y){}</code>
Lambda	<code>(a, b) -> a + b</code>
Try-with-resources	<code>try (var r = open()) { }</code>
Modern switch	<code>switch(x){ case A -> ...; }</code>

Cheat sheet – Collections & Streams

Need	Use
Ordered, resizable list	<code>ArrayList</code>
Unique elements	<code>HashSet</code>
Key-value lookup	<code>HashMap</code>
Thread-safe map	<code>ConcurrentHashMap</code>
Filter + transform + collect	<code>list.stream().filter(...).map(...).collect(Collectors.toList())</code>
Group by a field	<code>Collectors.groupingBy(...)</code>
Safe "may be absent"	<code>Optional<T></code>

Cheat sheet – SQL

Need	Syntax
Filter groups	GROUP BY x HAVING COUNT(*) > 1
Keep all left rows	LEFT JOIN ... ON ...
Named subquery	WITH cte AS (SELECT ...) SELECT * FROM cte
Rank within groups	RANK() OVER (PARTITION BY x ORDER BY y)
Check query plan	EXPLAIN ANALYZE SELECT ...

Cheat sheet – Spring / Spring Boot / JPA

Annotation	Purpose
@SpringBootApplication	Entry point: config + component scan + auto-config
@RestController / @RequestMapping	Declares a JSON-returning web controller and its base path
@Service / @Repository / @Component	Marks a class as a Spring-managed bean, by layer
@Autowired (constructor)	Injects a dependency – prefer constructor injection
@Entity / @Id / @GeneratedValue	Maps a class to a database table and its primary key
@OneToMany / @ManyToOne / @ManyToMany	Declares an entity relationship
@Transactional	Wraps a method in a database transaction
@Valid + @RequestBody	Validates an incoming JSON payload against a DTO's constraints
@RestControllerAdvice + @ExceptionHandler	Centralized error handling across all controllers

Cheat sheet – Git, Docker, Linux basics

Task	Command
Create + switch branch	git checkout -b feature/x
Replay commits cleanly	git rebase main
Build a Docker image	docker build -t app:1.0 .
Start a full local stack	docker compose up -d
Find what's using a port	lsof -i :8080
Tail logs live	tail -f app.log

Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability – the guarantees a database transaction provides
Bean	An object managed by the Spring IoC container
Circuit breaker	A pattern that stops calling a repeatedly failing dependency, failing fast instead
DTO	Data Transfer Object – shapes data crossing an API boundary, separate from the internal entity
Idempotent	An operation that produces the same result no matter how many times it's applied
JWT	JSON Web Token – a signed, self-contained token used for stateless authentication
N+1 problem	One query for a list of parents plus one more query per parent for related data
ORM	Object-Relational Mapping – mapping Java objects to database rows (Hibernate implements this)
Race condition	An outcome that depends on unpredictable thread timing due to unsynchronized shared state
SOLID	Five object-oriented design principles: Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion
TTL	Time To Live – how long a cached value stays valid before automatic expiry

The Java + Spring Boot Job-Ready Master Checklist

- | | |
|--|--|
| ☐ Core Java syntax, OOP, generics | ☐ Collections Framework, internal workings |
| ☐ Exception handling, custom exceptions | ☐ Java 8+ streams, lambdas, Optional |
| ☐ Multithreading, concurrency primitives | ☐ JVM internals, GC basics |
| ☐ HTTP, REST principles, status codes | ☐ SQL – joins, subqueries, window functions |
| ☐ JDBC fundamentals | ☐ Spring IoC/DI, bean lifecycle |
| ☐ Spring Boot auto-configuration | ☐ REST API design with Spring Boot |
| ☐ Spring Data JPA / Hibernate, N+1 fixes | ☐ Spring Security, JWT auth |
| ☐ Unit + integration testing | ☐ Microservices patterns & resilience |
| ☐ Kafka fundamentals | ☐ Redis caching |
| ☐ Docker & Docker Compose | ☐ CI/CD pipeline basics |
| ☐ Cloud fundamentals (AWS) | ☐ Observability: logs, metrics, tracing |
| ☐ System design method + 3 case studies practiced | ☐ Design patterns with real examples |
| ☐ 100+ DSA problems solved across difficulties | ☐ Mock interviews completed, all 5 tracks |
| ☐ Resume, GitHub, LinkedIn ready | ☐ 3+ portfolio projects + capstone complete |

END OF COURSE

This closes the 45-part curriculum. If you've worked through Parts 1–45 and completed the checklist above: you can build, test, secure, containerize, deploy, debug, and explain a production-grade Java + Spring Boot backend application. Return to the [Master Index](#) to revisit any part.

Part 32 – DSA for Java Backend Jobs

Big O · Core structures & techniques · Worked problems by pattern, with Java solutions

Big O – the vocabulary of efficiency

Notation	Name	Example
$O(1)$	Constant	HashMap.get()
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Scanning an array once
$O(n \log n)$	Linearithmic	Efficient sorting (merge sort)
$O(n^2)$	Quadratic	Nested loop comparing every pair
$O(2^n)$	Exponential	Naive recursive Fibonacci, brute-force subsets

Pattern: Two Pointers

```
// Problem: does a sorted array contain two numbers summing to target?
public boolean twoSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return true;
        else if (sum < target) left++;
        else right--;
    }
    return false;
}
// O(n) time, O(1) space – beats the brute-force O(n^2) nested loop
```

Pattern: Sliding Window

```
// Problem: longest substring without repeating characters
public int longestUniqueSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, sc-camel-max-len = 0;
    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left++));    // shrink from the left until valid again
        }
        window.add(s.charAt(right));
        sc-camel-max-len = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
// O(n) – each character is added and removed from the window at most once
```

Pattern: BFS/DFS on trees & graphs

```
public List<Integer> bfs(Map<Integer, List<Integer>> graph, int start) {
    List<Integer> order = new ArrayList<>();
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start); visited.add(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, List.of())) {
            if (visited.add(neighbor)) queue.offer(neighbor);
        }
    }
    return order;
}
// BFS explores level by level (shortest path in an unweighted graph);
// DFS (recursive or stack-based) explores as deep as possible before backtracking
```

Pattern: Dynamic Programming

```
// Problem: minimum coins to make amount (Coin Change)
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
// dp[i] = fewest coins to make amount i, built up from smaller subproblems – O(amount × coins)
```

THINK LIKE A DEVELOPER

Recognize DP by the question "what's the optimal way to reach state X, given the optimal ways to reach smaller states?" Always try the brute-force recursive solution first, spot the repeated subproblems, then convert to memoization (top-down) or a table (bottom-up, as above).

Pattern: Backtracking

```
// Problem: generate all subsets of a set
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}
private void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current)); // record every partial state
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]); // choose
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1); // un-choose (backtrack)
    }
}
// O(2^n) – explores every subset; the "choose, explore, un-choose" shape is the backtracking template
```

Pattern: Binary Search (on answer)

```
// Problem: find the minimum capacity to ship packages within `days` days
public int shipWithinDays(int[] weights, int days) {
    int lo = Arrays.stream(weights).max().getAsInt();
    int hi = Arrays.stream(weights).sum();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canShip(weights, days, mid)) hi = mid;    // mid works - try smaller
        else lo = mid + 1;                          // mid too small - need bigger
    }
    return lo;
}

// Binary search isn't just for sorted arrays - it works on any monotonic
// "does this value satisfy the condition?" function, which is a common
// interview twist once the basic array binary search is mastered.
```

Pattern: Intervals & Prefix Sum

```
// Merge overlapping intervals
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    for (int[] interval : intervals) {
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
            merged.add(interval);
        } else {
            merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() - 1)[1],
interval[1]);
        }
    }
    return merged.toArray(new int[0][]);
}

// Prefix sum - answer range-sum queries in O(1) after O(n) preprocessing
int[] prefix = new int[nums.length + 1];
for (int i = 0; i < nums.length; i++) prefix[i + 1] = prefix[i] + nums[i];
int sc-camel-range-sum = prefix[right + 1] - prefix[left];    // sum of nums[left..right]
```

Arrays & Hashing (1–8): 1) Group Anagrams. 2) Top K Frequent Elements. 3) Product of Array Except Self. 4) Longest Consecutive Sequence. 5) Contains Duplicate II (within distance k). 6) Subarray Sum Equals K. 7) Find All Duplicates in an Array. 8) Majority Element (Boyer-Moore voting).

Two Pointers & Sliding Window (9–16): 9) 3Sum. 10) Container With Most Water. 11) Minimum Window Substring. 12) Longest Repeating Character Replacement. 13) Permutation in String. 14) Sort Colors (Dutch flag). 15) Trapping Rain Water. 16) Max Consecutive Ones III.

Linked Lists (17–22): 17) Detect Cycle in a Linked List (Floyd's). 18) Find the Middle of a Linked List. 19) Merge K Sorted Lists. 20) Reorder List. 21) Add Two Numbers (as linked lists). 22) Copy List with Random Pointer.

Trees & BST (23–30): 23) Validate Binary Search Tree. 24) Lowest Common Ancestor of a BST. 25) Kth Smallest Element in a BST. 26) Serialize and Deserialize a Binary Tree. 27) Binary Tree Level Order Traversal. 28) Diameter of a Binary Tree. 29) Construct Binary Tree from Preorder and Inorder. 30) Balanced Binary Tree check.

Graphs (31–36): 31) Number of Islands. 32) Clone Graph. 33) Course Schedule (cycle detection via topological sort). 34) Word Ladder (shortest transformation via BFS). 35) Pacific Atlantic Water Flow. 36) Network Delay Time (Dijkstra).

Backtracking (37–41): 37) Permutations. 38) Combination Sum. 39) Word Search (grid DFS + backtrack). 40) N-Queens. 41) Palindrome Partitioning.

Dynamic Programming (42–50): 42) Climbing Stairs. 43) House Robber. 44) Longest Increasing Subsequence. 45) Longest Common Subsequence. 46) Edit Distance. 47) 0/1 Knapsack. 48) Partition Equal Subset Sum. 49) Unique Paths (grid DP). 50) Word Break.

Practice roadmap by difficulty

Level	Focus patterns	Example problems
Easy	Arrays, hashing, two pointers, basic recursion	Two Sum, Valid Parentheses, Reverse Linked List, Merge Two Sorted Lists, Maximum Subarray
Medium	Sliding window, BFS/DFS, binary search, backtracking, intervals	Longest Substring Without Repeats, Number of Islands, Course Schedule, Merge Intervals, Kth Largest Element
Hard	Advanced DP, graph algorithms, complex backtracking	Trapping Rain Water, Word Ladder, Edit Distance, Median of Two Sorted Arrays, N-Queens

INTERVIEW QUESTIONS – PART 32

1. Given an unsorted array, find two numbers that sum to a target – walk through both the $O(n^2)$ and $O(n)$ approaches.
2. When do you reach for BFS vs. DFS?
3. What signals that a problem is a sliding-window problem?
4. Explain memoization vs. tabulation in dynamic programming.
5. How do you detect a cycle in a directed graph?
6. Explain why binary search requires sorted (or monotonic) input.

Next: Part 33 – Java Coding Interview Preparation: core Java, OOP, collections, streams, and concurrency questions with strong answers.

Part 35 — Real-World Projects

Fifteen portfolio-ready backends — requirements, architecture, and the build checklist for each

Every project below follows the same production checklist: Controller → Service → Repository → Entity/DTO layering, request validation, centralized exception handling, JWT-secured endpoints where relevant, unit + integration tests, a Dockerfile, and a README documenting setup and API endpoints. Build each with the folder structure from Part 18, applying the specific domain concepts noted.

The fifteen projects

#	Project	Core entities	Key concepts exercised
1	Employee Management System	Employee, Department, Role	One-to-many relationships, RBAC authorization
2	Expense Tracker	Expense, Category, Budget	Aggregation queries, monthly reporting, pagination
3	Banking Application	Account, Transaction, Customer	@Transactional transfers, optimistic locking, audit trail
4	E-commerce Backend	Product, Cart, Order, Payment	Many-to-many (cart items), inventory decrement, order state machine
5	Learning Management System	Course, Lesson, Enrollment, Progress	Nested relationships, role-based content access
6	Job Portal	JobPosting, Application, Candidate, Recruiter	Full-text search/filtering, file upload (resumes) to S3
7	Food Delivery Backend	Restaurant, MenuItem, Order, Delivery	Geolocation queries, real-time status updates, Kafka events
8	Inventory Management System	Item, Warehouse, StockMovement	Concurrency control on stock updates, low-stock alerts
9	Payment Service	PaymentIntent, Refund, Ledger	Idempotency keys, webhook handling, reconciliation
10	Enterprise Microservices Platform	Spans multiple services from projects 1-9	API Gateway, service discovery, distributed tracing — ties together Parts 22-24
11	Hotel Booking System	Hotel, Room, Booking, Guest	Availability-window queries, double-booking prevention via constraints/locking
12	Social Media Backend	Post, Comment, Like, Follow	Self-referencing many-to-many (follows), feed pagination, fan-out on write vs. read
13	Healthcare Appointment System	Patient, Doctor, Appointment, Slot	Time-slot conflict validation, role-based views (patient vs. doctor vs. admin)

14	URL Shortener Service	ShortUrl, ClickEvent	Base62 encoding, Redis-backed redirect caching, click analytics via Kafka
15	Real-Time Chat Backend	Conversation, Message, Participant	WebSocket messaging, message delivery status, Redis pub/sub for multi-instance fan-out

Worked example: Expense Tracker, in outline

```
src/main/java/com/example/expensetracker/
├─ entity/ Expense.java, Category.java, Budget.java, User.java
├─ dto/ ExpenseRequest.java, ExpenseResponse.java, MonthlySummary.java
├─ repository/ ExpenseRepository.java (with @Query for monthly aggregation)
├─ service/ ExpenseService.java, BudgetAlertService.java
├─ controller/ ExpenseController.java, ReportController.java
├─ security/ SecurityConfig.java, JwtAuthFilter.java
├─ exception/ GlobalExceptionHandler.java
```

Key endpoint: GET /api/reports/monthly?month=2026-08
 → groups expenses by category, compares against Budget, flags overspend

```
@Query("SELECT new com.example.dto.CategoryTotal(e.category.name, SUM(e.amount)) " +
        "FROM Expense e WHERE e.user.id = :userId AND e.date BETWEEN :start AND :end " +
        "GROUP BY e.category.name")
List<CategoryTotal> sumByCategory(Long userId, LocalDate start, LocalDate end);
```

PRO TIP

Build these roughly in the listed order – each project reuses skills from the last while introducing exactly one or two new concepts (project 3 introduces transactional integrity; project 6 introduces file upload and search; project 9 introduces idempotency; project 15 introduces WebSockets from Part 46). Trying to combine everything in project 1 will slow you down.

INTERVIEW EXPLANATION PRACTICE

For each finished project, rehearse a 60-second verbal walkthrough: the problem it solves, the schema (draw the ER diagram from memory), one interesting technical decision (e.g. "I used optimistic locking on Account balance to prevent lost updates under concurrent transfers"), and one tradeoff you made under time constraints.

Next: Part 36 – The Capstone Project: one enterprise-style application combining everything.